



Hyper Separation Logic

TRAYAN GOSPODINOV, INSAIT, Sofia University “St. Kliment Ohridski”, Bulgaria

PETER MÜLLER, ETH Zurich, Switzerland

THIBAUT DARDINIER, ETH Zurich, Switzerland

Many important functional and security properties—including non-interference, determinism, and generalized non-interference (GNI)—are hyperproperties, i.e., properties relating multiple executions of a program. Existing separation logics allow one to reason about specific classes of hyperproperties, e.g., $\forall\forall$ -hyperproperties such as non-interference and $\exists\exists$ -properties such as non-determinism. However, they do not support quantifier alternation, which is for instance needed to express GNI. The only existing logic that can reason about such properties is Hyper Hoare Logic, but it does not support heap-manipulating programs and, thus, is not applicable to common imperative programs.

This paper introduces Hyper Separation Logic (HSL), the first program logic that supports modular reasoning about hyperproperties with arbitrary quantifier alternation over programs that manipulate the heap. HSL generalizes Hyper Hoare Logic with a novel hyper separating conjunction that lifts the standard separating conjunction to sets of states, enabling a generalized frame rule for hyperproperties. We prove HSL sound in Isabelle/HOL and demonstrate its expressiveness for hyperproperties that lie beyond the reach of existing separation logics.

CCS Concepts: • **Theory of computation** → **Logic and verification**; **Separation logic**; **Hoare logic**.

Additional Key Words and Phrases: Hyperproperties, Program Logic, Incorrectness Logic, Generalized Non-Interference, Hyper Separating Conjunction, Labeled Separating Conjunction

ACM Reference Format:

Trayan Gospodinov, Peter Müller, and Thibault Dardinier. 2026. Hyper Separation Logic. *Proc. ACM Program. Lang.* 10, PLDI, Article 250 (June 2026), 24 pages. <https://doi.org/10.1145/3808328>

1 Introduction

Many important functional and security properties are *hyperproperties* [8], i.e., properties that relate *multiple* executions of a program. For example, information flow security, which requires that confidential inputs do not leak through public outputs, can be formalized as *non-interference* (NI) [21]: NI requires that any two executions with the same public inputs (but potentially different confidential inputs) produce the same public outputs. NI is a $\forall\forall$ -hyperproperty (also called 2-*hypersafety*), i.e., a property of all *pairs* of executions. Important functional hyperproperties include determinism ($\forall\forall$), transitivity ($\forall\forall\forall$, important for custom comparators [36]), or associativity ($\forall\forall\forall\forall$, important for parallel reductions).

Many hyperproperties, however, fall outside the class of $\forall^k$ -hyperproperties because they require existential quantification over executions. For example, reachability is an $\exists$ -hyperproperty, as it asserts the existence of an execution that reaches a given state, while non-determinism is an $\exists\exists$ -hyperproperty, requiring two executions with the same inputs that produce different outputs. More expressive hyperproperties require *quantifier alternation* (i.e., $\forall^*\exists^*$ or $\exists^*\forall^*$). An important example

Authors’ Contact Information: [Trayan Gospodinov](mailto:Trayan.Gospodinov@insait.ai), INSAIT, Sofia University “St. Kliment Ohridski”, Sofia, Bulgaria, trayan.gospodinov@insait.ai; [Peter Müller](mailto:Peter.Mueller@inf.ethz.ch), ETH Zurich, Zurich, Switzerland, peter.mueller@inf.ethz.ch; [Thibault Dardinier](mailto:Thibault.Dardinier@inf.ethz.ch), ETH Zurich, Zurich, Switzerland, thibault.dardinier@inf.ethz.ch.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2475-1421/2026/6-ART250

<https://doi.org/10.1145/3808328>

is *generalized non-interference* (GNI) [27], a weakening of NI for non-deterministic programs. GNI is a $\forall\exists$ -hyperproperty: it permits two executions τ_1 and τ_2 with the same low inputs to have different low outputs, provided that there exists a third execution τ with the same low inputs, the same high inputs as τ_1 , and the same low outputs as τ_2 . Hyperproperties with $\exists^*\forall^*$ prefixes are also important, for example to express the existence of a minimum or a violation of GNI.

As reasoning about hyperproperties has gained importance, many Hoare-like program logics for hyperproperties have been proposed. Relational Hoare Logic [4] extends Hoare Logic [17, 24] to $\forall\forall$ -hyperproperties, and has later been extended to $\forall^k$ -hyperproperties [14, 36]. A complementary line of work has developed program logics for $\exists$ [12, 32] and $\exists\exists$ [29] hyperproperties, targeting reachability and incorrectness. Recent work has either unified safety and incorrectness reasoning in a single logic [38], or has developed logics for $\forall^*\exists^*$ -hyperproperties [1, 5, 13] (such as GNI). The only program logic that handles arbitrary quantifier alternation is Hyper Hoare Logic (HHL) [9].

None of these logics apply to heap-manipulating programs. Separation Logic (SL) [35] supports reasoning about such programs in a modular, local way via its *frame rule*. Thus, unsurprisingly, many Hoare logics for hyperproperties have been extended to support SL reasoning. Separation logics for $\forall^k$ -hyperproperties include Relational Separation Logic [37] (for $k = 2$), and LGTM [20] (for unbounded k), while separation logics for $\exists^k$ -hyperproperties include Incorrectness Separation Logic [33, 34] and Sufficient Incorrectness Separation Logic [2] (for $k = 1$), and InsecSL [30] (for $k = 2$). Additionally, Outcome Separation Logic [39] and Exact Separation Logic [25] unify safety and reachability reasoning for *single* executions, i.e., they support a mix of $\forall$ and $\exists$ properties.

However, no existing program logic supports reasoning about $\forall^+\exists^+$, $\exists^+\forall^+$, or even $\exists^k$ (for $k > 2$) hyperproperties for heap-manipulating programs. Without a logic that supports such properties, it is extremely difficult to prove useful properties such as GNI for realistic imperative programs.

This work. We present *Hyper Separation Logic* (HSL), the first program logic that supports proving hyperproperties with arbitrary quantifier alternation for heap-manipulating programs. HSL builds on the foundations of HHL: It establishes *hyper-triples* of the form $[P] \ C \ [Q]$, where P and Q are *hyper-assertions*, i.e., arbitrarily-quantified predicates over sets of states (including a heap). HSL supports local, modular reasoning via a novel *generalized frame rule* for hyper-assertions: From a local triple $[P] \ C \ [Q]$, one can derive a new triple $[P \star F] \ C \ [Q \star F]$ (under suitable side conditions), which can be applied in a larger context. Here, $\star$ is a novel *hyper separating conjunction*, which generalizes the separating conjunction $*$ from SL. Intuitively, the hyper-assertion $P \star F$ expresses that a set S of states can be, per state, split into two sets of states S_P (satisfying P) and S_F (satisfying F), such that each state in S is a disjoint union of a state in S_P and a state in S_F .

Contributions. Our main contributions are:

- We present *Hyper Separation Logic* (HSL), the first separation logic for hyperproperties with arbitrary quantifier alternation.
- We introduce a novel *hyper separating conjunction* connective between hyper-assertions, which preserves upper bounds (for universally quantified executions) and lower bounds (for existentially quantified executions) of sets of states.
- We introduce a novel definition of hyper-triple validity, which guarantees soundness of the generalized frame rule by construction. Our definition is based on a novel decomposition of $\forall^*$ -hyperproperties into $\forall$ -properties.
- To support reachability reasoning, we introduce a novel notion of *unknown* states, and weaken the definition of hyper-triple validity accordingly.
- We introduce and prove sound a proof system for HSL, including rules for basic heap-manipulating commands, a generalized frame rule, and rules to reason about errors.

- To ease reasoning, we introduce a syntax for hyper-assertions, as well as a novel notion of *scaffold* variables, and use the latter to derive a more expressive rule to read from the heap.
- Finally, we showcase our logic on a diverse set of examples, including for $\forall^*\exists^*$ and $\exists^*\forall^*$ hyperproperties, which are beyond the reach of existing separation logics.

All formal results about HSL presented in this paper have been formalized in Isabelle/HOL [31], and our mechanization is publicly available [22].

Outline. Sect. 2 gives a high-level overview of some of the main novelties of HSL, in particular its hyper-triples, hyper separating conjunction, and generalized frame rule. Sect. 3 then presents our solutions to key technical challenges, including how we define the hyper separating conjunction, how we ensure the soundness of the frame rule with a novel decomposition of $\forall^*$ -hyperproperties, and how our unknown states enable reachability reasoning. Sect. 4 presents HSL formally, while we discuss related work in Sect. 5, and conclude in Sect. 6.

2 A Tour of Hyper Separation Logic

2.1 Hyper-Triples: Hyperproperties and Ownership

HSL's judgments are *hyper-triples* of the form $[P] C [Q]$, where C is a program statement, and P and Q are *hyper-assertions*, i.e., predicates over sets of states. Intuitively, the hyper-triple $[P] C [Q]$ holds iff for every set of initial states S satisfying P , the set S' of states reachable by executing C in any state from S satisfies Q . HSL judgments can express arbitrary program hyperproperties, that is, properties relating the initial and final states of multiple executions of a program, such as determinism, generalized non-interference, and the existence of a leak of confidential data. Like HHL, HSL obtains this expressiveness by allowing hyper-assertions to contain arbitrary alternations of universal and existential quantification over states.

However, HSL goes substantially beyond HHL by supporting heap-manipulating programs. In contrast to HHL, states in HSL contain a heap, and HSL assertions may express properties about the heap using standard SL operators, in particular, the points-to predicate $\mapsto$ and separating conjunction $*$. For example, the triple

$$[\forall\langle\sigma\rangle. \sigma(x \mapsto 5 * y \mapsto _)] [y] := [x] + 1 [\forall\langle\sigma\rangle. \sigma(x \mapsto 5 * y \mapsto 6)]$$

is equivalent to the standard SL triple $\{x \mapsto 5 * y \mapsto _ \} [y] := [x] + 1 \{x \mapsto 5 * y \mapsto 6\}$. Both the pre- and the postcondition quantify universally over all states, thereby expressing a property of a single execution of the assignment (we will show examples of proper hyperproperties below). The precondition expresses that all initial states σ own the heap locations x (containing the value 5) and y . Since HSL assertions generally relate states from multiple executions, assertions explicitly indicate which state they refer to, as in $\sigma(\dots)$. The postcondition expresses that all reachable states own the same locations, and that the value stored at location y is now 6. The standard separating conjunction $*$ enforces that x and y point to disjoint parts of the heap, such that x 's value is not affected by the assignment. As we will see in the next subsection, at the core of HSL lies a hyper separating conjunction $\star$, which generalizes the standard $*$ to express properties of an entire set of heaps rather than a single heap.

The hyper-triple $[\exists\langle\sigma\rangle. \sigma(p)] C [\exists\langle\sigma\rangle. \sigma(q)]$ corresponds to the sufficient separation incorrectness logic (SSIL) [3] triple $\{p\} C \{q\}$, which expresses that executing C in any state satisfying p can (non-deterministically) reach a state satisfying q . For example, the hyper-triple

$$[\exists\langle\sigma\rangle. \sigma(r \mapsto 1)] x := [r]; y := \text{randInt}(1, 6); z := x + y [\exists\langle\sigma\rangle. \sigma(r \mapsto 1 * z = 6)]$$

expresses that executing the statement in an initial state with heap location r allocated and set to 1, can reach a final state satisfying additionally $z = 6$.

A key use case of reachability reasoning as in the previous example is to prove the presence of a bug. To this end, following Incorrectness [32] and Outcome Logic [38], our assertions distinguish *normal states* σ , for which we write $\sigma(\text{ok} : \dots)$, from *error states*, for which we write $\sigma(\text{er} : \dots)$. For example, the hyper-triple

$$[\exists \langle \sigma \rangle. \sigma(\text{ok} : r \mapsto _)] \text{ free}(r); \text{ free}(r) [\exists \langle \sigma \rangle. \sigma(\text{er} : \top)]$$

expresses the reachability of an error state, that is, the presence of a bug. For this statement, one could also write a stronger triple expressing that all executions *will* fail, by weakening the precondition and using universal quantification in the postcondition. To avoid clutter, we write $\sigma(p)$ short for $\sigma(\text{ok} : p)$.

Expressing hyperproperties. Let us now illustrate how HSL can express hyperproperties that relate values stored on the heap. The following triple expresses that the function *split*, which takes as input one pointer and returns two pointers, is deterministic (i.e., two calls with the same value at heap location l will result in the same values at heap locations x and y):

$$\begin{aligned} & [\forall \langle \sigma_1 \rangle. \forall \langle \sigma_2 \rangle. \exists u. \sigma_1(l \mapsto u) \wedge \sigma_2(l \mapsto u)] \\ & \quad x, y := \text{split}(l) \\ & [\forall \langle \sigma_1 \rangle. \forall \langle \sigma_2 \rangle. \exists v, w. \sigma_1(x \mapsto v * y \mapsto w) \wedge \sigma_2(x \mapsto v * y \mapsto w)] \end{aligned}$$

The precondition expresses that all initial states have ownership of the heap location l , and that all states have the same value u stored at location l . The postcondition then expresses that all reachable states have ownership of the two returned pointers x and y , with the same values v and w , resp. Thus, these output values depend only on the input stored at l .

A key feature of HSL is that universal and existential quantification can be combined to express, for instance, $\forall^* \exists^*$ and $\exists^* \forall^*$ hyperproperties. An important example is generalized non-interference (GNI) [26, 28], a standard notion of secure information flow for non-deterministic programs which, to the best of our knowledge, no other separation logic can express. GNI requires that for any two executions that agree on the public (low-sensitivity) inputs, there exists a third execution with the same low inputs that has the same high inputs as the first execution and the same low outputs as the second execution. The following triple expresses that the function *compute*, which takes two pointers x (low) and h (high) and returns one pointer o , satisfies GNI:

$$\begin{aligned} & [\forall \langle \sigma_1 \rangle. \forall \langle \sigma_2 \rangle. \exists v. \sigma_1(x \mapsto v * h \mapsto _) \wedge \sigma_2(x \mapsto v * h \mapsto _)] \\ & \quad o := \text{compute}(x, h) \\ & [\forall \langle \sigma_1 \rangle. \forall \langle \sigma_2 \rangle. \exists \langle \sigma \rangle. \exists u, v. \sigma_1(o \mapsto _ * h \mapsto v) \wedge \sigma(o \mapsto u * h \mapsto v) \wedge \sigma_2(o \mapsto u * h \mapsto _)] \end{aligned}$$

In this triple, we assume for simplicity that *compute* does not modify the value stored in h ; this property could be expressed using logical variables, as usual [9].

HSL can express and reason about arbitrary program hyperproperties, such as the examples above, uniformly within one logic. This expressiveness allows one in particular to combine different program hyperproperties within the same proof, a flexibility that other, more specialized logics do not offer. In the rest of this section, we will show that HSL also enables uniform separation-logic reasoning across different program hyperproperties.

2.2 Hyper Separating Conjunction

At the core of HSL is the *hyper separating conjunction* $\star$, which generalizes standard separating conjunction to hyper-assertions. Hyper separating conjunction allows one to (de)compose the heaps of all states in a set of states uniformly for both ok and error states and for both universally- and existentially-quantified states. Intuitively, a set of states S satisfies the hyper-assertion $P \star Q$ iff there exist two (possibly overlapping) sets $S_P \models P$ and $S_Q \models Q$ such that each state $\sigma \in S$ can be partitioned into two states $\sigma_p \in S_P$ and $\sigma_q \in S_Q$ whose heaps are disjoint.

Hyper separating conjunction lifts separation logic reasoning to sets of states, which enables concise proofs of program hyperproperties, as HSL's rule for memory allocation illustrates:

$$\text{ALLOC} \quad \frac{\text{no } \sigma(\text{er} : _) \text{ in } P \quad x \notin \text{fv}(P)}{\models [P] \ x := \text{alloc}() \ [P \star (\forall \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto _))]}$$

This rule expresses that allocation preserves any hyperproperty P and that all reachable states additionally have ownership of the newly-allocated location x (we will discuss in Sect. 4.5 why the preservation of P is built into the rule). The two premises ensure that P does not mention error states (which we will motivate later) or x (such that P is not affected by the assignment).

The hyper separating conjunction in the postcondition elegantly composes the heap of each state constrained by P with the newly-allocated memory. As an example, consider the precondition of the function *split* discussed above, that is, $P \triangleq \forall \langle \sigma_1 \rangle. \forall \langle \sigma_2 \rangle. \exists u. \sigma_1(l \mapsto u) \wedge \sigma_2(l \mapsto u)$. Using laws (formally proven in our mechanization), we can distribute the new ownership of x over the quantified states σ_1 and σ_2 as follows:

$$\begin{aligned} & (\forall \langle \sigma_1 \rangle. \forall \langle \sigma_2 \rangle. \exists u. \sigma_1(l \mapsto u) \wedge \sigma_2(l \mapsto u)) \star (\forall \langle \sigma \rangle. \sigma(x \mapsto _)) \\ &= \forall \langle \sigma_1 \rangle. \forall \langle \sigma_2 \rangle. \exists u. (\sigma_1(l \mapsto u * x \mapsto _) \wedge (\sigma_2(l \mapsto u * x \mapsto _))) \end{aligned}$$

On the other hand, if P quantified existentially over states, for instance, $P \triangleq \exists \langle \sigma \rangle. \sigma(y \mapsto 1)$ then the postcondition would entail $\exists \langle \sigma \rangle. \sigma(y \mapsto 1 * x \mapsto _)$, correctly expressing the reachability of a state owning both y and x . These examples illustrate that $\star$ expresses the intended property uniformly for different quantifiers in the conjuncts. We will see in Sect. 3 how it achieves that.

2.3 Local Reasoning and Framing

A key benefit of separation logic is that properties can be proven *locally* considering only the relevant portion of the heap and then applied in a larger context via the frame rule. HSL supports local reasoning via the following *generalized frame rule*, which lifts framing to sets of states:

$$\text{FRAME} \quad \frac{\models [P] \ C \ [Q] \quad \text{No } \exists \langle _ \rangle \text{ in } F \quad F \models \forall \langle \sigma \rangle. \sigma(\text{ok} : _) \quad \text{fv}(F) \cap \text{md}(C) = \emptyset}{\models [P \star F] \ C \ [Q \star F]}$$

Analogously to the standard frame rule, this rule allows us to extend a valid triple $\models [P] \ C \ [Q]$ with a *frame* F , which describes properties unaffected by C . However, in contrast to the standard frame rule, the assertions P , Q , and F in our generalized frame rule are all hyper-assertions and combined using our hyper separating conjunction, enabling framing in proofs of hyperproperties. This rule has three¹ side conditions to ensure soundness. First, the frame F is not allowed to contain existential quantification over states, as such existential quantifiers in the postcondition would guarantee reachability and thus termination, while C is not guaranteed to terminate in general.² Second, all states described by the frame must be ok states; this condition is explained when defining validity (Sect. 4.4). The third restriction is standard: it ensures the frame's free variables (fv) aren't modified (md) by the command C .

The example in Fig. 1 illustrates our frame rule by proving that the program in black satisfies generalized non-interference (GNI), a property that is beyond the reach of existing separation logics due to its quantifier alternation in the postcondition. We assume the value at input location l has low-sensitivity (public) and the value at input location h has high-sensitivity (confidential). Analogously to *compute* above, we verify the following triple:

¹As we will see in Sect. 4.7, we need a fourth restriction for assertions containing *scaffold* variables.

²We believe that this restriction could be lifted for terminating programs.

```

(1)    [∀⟨σ1⟩. ∀⟨σ2⟩. ∃u. σ1(l ↦ u * h ↦ _) ∧ σ2(l ↦ u * h ↦ _)]
(2)    ⊨ [(∀⟨σ1⟩. ∀⟨σ2⟩. ∃u. σ1(l ↦ u) ∧ σ2(l ↦ u)) * (∀⟨σ⟩. σ(h ↦ _))]
(3)    [∀⟨σ1⟩. ∀⟨σ2⟩. ∃u. σ1(l ↦ u) ∧ σ2(l ↦ u)]
(4)    x, y := split(l)
(5)    [∀⟨σ1⟩. ∀⟨σ2⟩. ∃v, w. σ1(x ↦ v * y ↦ w) ∧ σ2(x ↦ v * y ↦ w)]
(6)    [(∀⟨σ1⟩. ∀⟨σ2⟩. ∃v, w. σ1(x ↦ v * y ↦ w) ∧ σ2(x ↦ v * y ↦ w)) * (∀⟨σ⟩. σ(h ↦ _))]
(7)    ⊨ [∀⟨σ1⟩. ∀⟨σ2⟩. ∃v, w. σ1(x ↦ v * y ↦ w * h ↦ _) ∧ σ2(x ↦ v * y ↦ w * h ↦ _)]
(8)    ⊨ [(∀⟨σ1⟩. ∀⟨σ2⟩. ∃v. σ1(x ↦ v * h ↦ _) ∧ σ2(x ↦ v * h ↦ _)) * (∀⟨σ1⟩. ∀⟨σ2⟩. ∃w. σ1(y ↦ w) ∧ σ2(y ↦ w))]
(9)    [∀⟨σ1⟩. ∀⟨σ2⟩. ∃v. σ1(x ↦ v * h ↦ _) ∧ σ2(x ↦ v * h ↦ _)]
(10)   o := compute(x, h)
(11)   [∀⟨σ1⟩. ∀⟨σ2⟩. ∃⟨σ⟩. ∃u, v. σ1(o ↦ _ * h ↦ v) ∧ σ(o ↦ u * h ↦ v) ∧ σ2(o ↦ u * h ↦ _)]
(12)   [(∀⟨σ1⟩. ∀⟨σ2⟩. ∃⟨σ⟩. ∃u, v. σ1(o ↦ _ * h ↦ v) ∧ σ(o ↦ u * h ↦ v) ∧ σ2(o ↦ u * h ↦ _)) * (∀⟨σ1⟩. ∀⟨σ2⟩. ∃w. ...)]
(13)   ⊨ [∀⟨σ1⟩. ∀⟨σ2⟩. ∃⟨σ⟩. ∃u, v, w. σ1(h ↦ u * o ↦ _ * y ↦ _) ∧ σ(h ↦ u * o ↦ v * y ↦ w) ∧ σ2(h ↦ _ * o ↦ v * y ↦ w)]
(14)   r := [o] + [y]
(15)   [∀⟨σ1⟩. ∀⟨σ2⟩. ∃⟨σ⟩. ∃u, v. σ1(h ↦ u) ∧ σ(h ↦ u ∧ r = v) ∧ σ2(h ↦ _ ∧ r = v)]

```

Fig. 1. Local reasoning with hyper-triples. The proof contains two applications of our generalized frame rule, one with a $\forall$ -hyperproperty and one with a $\forall\forall$ -hyperproperty.

$$\begin{aligned}
& [\forall\langle\sigma_1\rangle. \forall\langle\sigma_2\rangle. \exists u. \sigma_1(l \mapsto u * h \mapsto _) \wedge \sigma_2(l \mapsto u * h \mapsto _)] \\
& x, y := \text{split}(l); o := \text{compute}(x, h); r := [o] + [y] \\
& [\forall\langle\sigma_1\rangle. \forall\langle\sigma_2\rangle. \exists\langle\sigma\rangle. \exists u, v. \sigma_1(h \mapsto u) \wedge \sigma(h \mapsto u \wedge r = v) \wedge \sigma_2(h \mapsto _ \wedge r = v)]
\end{aligned}$$

Step 1: Framing a $\forall$ -property to verify the first call. To verify the call to *split*, we need to frame the ownership of the pointer h via the frame rule: We do so by rewriting the precondition on line 1 into the form on line 2, where we separate the part needed for calling *split* (line 3) from the part we frame, $(\forall\langle\sigma\rangle. \sigma(h \mapsto _))$. This allows us to use the specification of *split* (lines 3–5) to obtain the postcondition on line 6, where we add back the framed part $(\forall\langle\sigma\rangle. \sigma(h \mapsto _))$. This use of the frame rule is similar to the one in standard SL. By distributing the frame over the conjuncts of the postcondition (line 7), we get that each execution has ownership of x , y , and h , and that the values stored at x and y are the same across all executions.

Step 2: Framing a $\forall\forall$ -hyperproperty to verify the second call. The call to *compute* does not need ownership of y , as it accesses only x and h . As before, we could extract ownership to y via the frame $(\forall\langle\sigma\rangle. \sigma(y \mapsto _))$, but this would lose the fact that all executions have the same value stored at y . To preserve this information, we use the frame $\forall\langle\sigma_1\rangle. \forall\langle\sigma_2\rangle. \exists w. \sigma_1(y \mapsto w) \wedge \sigma_2(y \mapsto w)$ instead, which is supported by our generalized frame rule. Lines 9–11 then use the specification of *compute* to obtain the postcondition after the call, and line 12 adds back the frame to the postcondition. As before, lines 12–13 distribute the frame over the conjuncts of the postcondition.

Step 3: Reading from the heap. The postcondition follows from line 13 and the read operation. In particular, since the values stored in o and y are the same in σ and σ_2 , their sum r is also the same.

This example demonstrates that HSL can verify complex hyperproperties, such as GNI with its combination for universal and existential quantification. It combines $\forall$, $\forall\forall$, and $\forall\forall\exists$ -hyperproperties within the same proof, demonstrating the logic’s versatility. In particular, the generalized frame rule is used to frame both simple ownership information and hyperproperties. In the next section, we survey the key technical innovations that enable this flexibility, before formalizing the logic.

3 Key Technical Ingredients

3.1 Hyper Separating Conjunction

Existing separation logics for hyperproperties define separating conjunction pointwise over the elements of a(n ordered) k -tuple. This simple definition does not carry over to HSL, where assertions are expressed over (unordered) sets of states.

To motivate our definition of hyper separating conjunction, consider the hyper-assertion

$$(\forall \langle \sigma \rangle. \sigma(x \mapsto 5)) \star (\exists \langle \sigma \rangle. \sigma(y \mapsto 5)) \quad (1)$$

Intuitively, it should express that (1) both $\forall \langle \sigma \rangle. \sigma(x \mapsto 5)$ and $\exists \langle \sigma \rangle. \sigma(y \mapsto 5)$ hold (that is, act as a conjunction), and (2) the resources denoted by the conjuncts are disjoint (that is, be separating). In other words, in any set of states satisfying this hyper-assertion, all states should satisfy $x \mapsto 5$, and at least one state should satisfy $x \mapsto 5 * y \mapsto 5$.

This example demonstrates that the hyper separating conjunction needs to preserve both the *upper bounds* on the set of states expressed by each conjunct (via universal quantification, as in our first conjunct) and the *lower bounds* expressed via existential quantification, as in our second conjunct, ensuring the existence of at least one *witness state* satisfying the existential condition. We achieve both with the following definition:

$$S \models P \star Q \stackrel{\text{def}}{\iff} \exists S_P \models P. \exists S_Q \models Q. S \subseteq S_P * S_Q \wedge \text{pw}(S, S_P, S_Q) \quad (2)$$

Preserving upper bounds. The first conjunct of the definition ensures that the upper bounds expressed by P and Q are preserved: For sets of states S_P and S_Q that satisfy P and Q , respectively, any set of states S satisfying the hyper separating conjunction contains *at most* the states in the (standard) separating conjunction of S_P and S_Q ³. That is, each state $\sigma \in S$ can be decomposed into $\sigma = \sigma_P \oplus \sigma_Q$ such that $\sigma_P \in S_P$ and $\sigma_Q \in S_Q$, where $\oplus$ denotes the standard state-combining predicate from separation logic: $\sigma = \sigma_P \oplus \sigma_Q$ holds whenever the three program states agree on the same program store, and the heap component of σ results from combining the disjoint heaps of σ_P and σ_Q .

Due to the first conjunct in (2), every state in a set of states satisfying the hyper-assertion (1) includes $x \mapsto 5$. Moreover, every set of states satisfying the hyper-assertion $(\forall \langle \sigma \rangle. \sigma(x \mapsto 5)) \star (\forall \langle \sigma \rangle. \sigma(y \mapsto 5))$ includes only states with $x \neq y$. Consequently, $(\forall \langle \sigma \rangle. \sigma(x \mapsto 5)) \star (\forall \langle \sigma \rangle. \sigma(x \mapsto 5))$ is satisfied only by the empty set. These examples demonstrates that hyper separating conjunction implies non-aliasing, thereby preserving one of the key properties of separating conjunction.

However, the first conjunct of our definition (2) does *not* preserve the lower bounds of the conjuncts. For instance, the empty set of states satisfies it for the hyper-assertion (1) (by choosing the empty set for S_P), but does not include the witness state required by the second conjunct of (1).

Preserving lower bounds. The purpose of the second conjunct $\text{pw}(S, S_P, S_Q)$ of our definition (2) is to preserve the lower bounds expressed by P and Q . It consists of two symmetrical conjuncts, which we abbreviate as $\text{plw}(S, S_P, S_Q)$ and $\text{prw}(S, S_P, S_Q)$ ⁴. This conjunct addresses the lower bound, ensuring that the states from S_P and S_Q witnessing the (potential) existential quantifiers from P and Q , respectively, are preserved in S :

$$\begin{aligned} \text{pw}(S, S_P, S_Q) &\stackrel{\text{def}}{\iff} (\forall \sigma_P \in S_P. \exists \sigma_Q \in S_Q. \exists \sigma \in S. \sigma = \sigma_P \oplus \sigma_Q) && (\text{plw}(S, S_P, S_Q)) \\ &\quad \wedge (\forall \sigma_Q \in S_Q. \exists \sigma_P \in S_P. \exists \sigma \in S. \sigma = \sigma_P \oplus \sigma_Q) && (\text{prw}(S, S_P, S_Q)) \end{aligned}$$

³Our definition treats the sets of states S_P and S_Q as assertions, which allows us to apply separating conjunction.

⁴ pw , plw , and prw stand for “preserve witnesses”, “preserve left witnesses”, and “preserve right witnesses”, respectively.

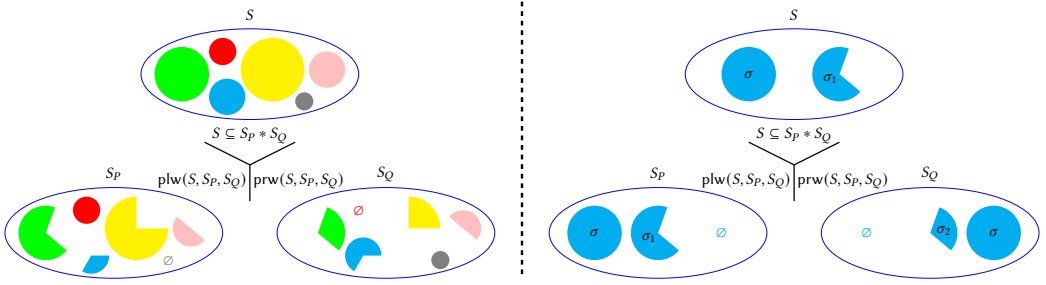


Fig. 2. Illustrations of two applications of the hyper separating conjunction. Circular sectors denote the heap component of states; identical (resp. different) colors indicate identical (resp. different) store components). $\emptyset$ denotes a state with empty heap.

The first preservation conjunct, plw , guarantees that all states σ_P (in particular, the witness states) from S_P have a corresponding state $\sigma \in S$; prw ensures a symmetric preservation for the states of S_Q . In our example, prw ensures that any set S satisfying the hyper-assertion (1) contains a state σ satisfying $x \mapsto 5 * y \mapsto 5$.

While the preservation predicates are not necessary for the soundness of the logic, they play a pivotal role in enabling reachability reasoning. In their absence, rules involving $\star$ —such as `ALLOC`—would fail to propagate witness information. In particular, for any satisfiable P , the postcondition, $P \star (\forall \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto _))$, would be trivially satisfied by $\emptyset$, eliminating any reachability guarantees.

Examples. Fig. 2 provides further intuition for our definition (2) of hyper separating conjunction. The diagram on the left shows a simple instance of the hyper separating conjunction, where each state in S corresponds to exactly one composition of a state in S_P and a state in S_Q , thereby satisfying both conjuncts of the definition. The diagram on the right shows more subtle cases. In particular, a state $\sigma \in S$ may be obtained in multiple ways ($\sigma = \sigma \oplus \emptyset$, $\sigma = \sigma_1 \oplus \sigma_2$ and $\sigma = \emptyset \oplus \sigma$). Moreover, the states in S_P (and S_Q) may participate in multiple combinations ($\sigma = \sigma_1 \oplus \sigma_2$ and $\sigma_1 = \sigma_1 \oplus \emptyset$). Note that in the right diagram, multiple sets S satisfy definition (2) for the given S_P and S_Q ; for example, extending the depicted S by $\emptyset$ still satisfies all requirements. In contrast, in the left diagram, S is uniquely determined by the given S_P and S_Q . In particular, adding $\emptyset$ to S would violate the first conjunct of definition (2) because the two $\emptyset$ states have different stores and therefore cannot be combined.

So far, we have illustrated hyper separating conjunction on conjuncts with a single quantifier, but our definition supports arbitrary quantifier alternations. For example, each set of states S_P satisfying the first conjunct of the hyper-assertion

$$(\forall \langle \sigma \rangle. \exists \langle \sigma' \rangle. \exists n. \sigma(x = n) \wedge \sigma'(x = n + 1)) \star (\forall \langle \sigma \rangle. (x \mapsto _))$$

is either empty or contains infinitely many states (with increasing values of x). Definition (2) retains these infinitely many witness states (via plw) in S and ensures (via its first conjunct) that each state in S owns the heap location at that particular address.

Labeled States. Thus far, we have presented the (hyper) separating conjunction for unlabeled states. In the following, we adapt the definition to HSL’s labeled states (see Sect. 2.1). Existing solutions do not apply to our setting. Raad et al. [33] treat the label merely as a tag in the postcondition while applying the separating conjunction to unlabeled states; this approach does not apply to our hyper-assertions, which constrain sets of states with possibly different labels. Zilberstein et al. [39] use an asymmetric definition where only one argument carries labels; in contrast, we aim for our

hyper separating conjunction to be symmetric, which is especially important for future extensions to concurrency.

We define both standard and hyper separating conjunctions by lifting the underlying state-combining predicate $\oplus$ from unlabeled to labeled states. $(\sigma)_\epsilon = (\sigma_1)_k \oplus (\sigma_2)_l$ holds iff (1) $\sigma = \sigma_1 \oplus \sigma_2$ and (2) $\epsilon = \text{ok}$ iff $k = \text{ok}$ and $l = \text{ok}$. The case that all three labels are ok corresponds to the standard, unlabeled program states. As a direct consequence, we obtain that $(\text{ok} : p) * (\text{ok} : q) = \text{ok} : p * q$, where we overload $*$ to denote both labeled and unlabeled separating conjunctions.

To understand the intuition for the case $k = \text{er}, l = \text{ok}$, consider an execution that starts in an ok-state and results in an error state $(\sigma_1)_k$, for instance, by causing a runtime error. Extending both the initial and the final state with the ok-state $(\sigma_2)_l$ will not eliminate the runtime error, so the extended final state is still an error state⁵. The case $k = \text{ok}, l = \text{er}$ is symmetric. As a direct consequence, we obtain that $(\text{er} : p) * (\text{ok} : q) = (\text{ok} : p) * (\text{er} : q) = \text{er} : p * q$. A similar argument justifies that combining two error states yields an error state and hence $(\text{er} : p) * (\text{er} : q) = \text{er} : p * q$.

3.2 One for All: Enforcing $\forall$ -Frames

Let us now define the validity of hyper-triples. As discussed in Sect. 2, intuitively the triple $[P] C [Q]$ is valid iff for every set of states S satisfying the precondition P , executing C from each state in S yields a set of states satisfying the postcondition Q . We denote this *candidate* validity $\models_0 [P] C [Q]$.

Sound framing. Unfortunately, this candidate definition does not support the frame rule, as shown by the following *invalid* example:

$$\frac{\begin{array}{c} \models_0 [\exists \langle \sigma \rangle. \sigma(\text{emp})] \ x := \text{alloc}() \ [\exists \langle \sigma \rangle. \sigma(x \mapsto _ * x = 42)] \\ \hline \not\models_0 \underbrace{[(\exists \langle \sigma \rangle. \sigma(\text{emp})) \star (\forall \langle \sigma \rangle. \sigma(42 \mapsto _))]}_{\supset (\exists \langle \sigma \rangle. \sigma(42 \mapsto _)) \wedge (\forall \langle \sigma \rangle. \sigma(42 \mapsto _))} \ x := \text{alloc}() \ \underbrace{[(\exists \langle \sigma \rangle. \sigma(x \mapsto _ * x = 42)) \star (\forall \langle \sigma \rangle. \sigma(42 \mapsto _))]}_{\perp} \end{array}}_{\text{FRAME}}$$

According to our candidate definition, the first triple is valid: An execution starting with an empty heap might (non-deterministically) allocate address 42, yielding a state satisfying the postcondition. However, this specific address is ruled out when combined with the frame $\forall \langle \sigma \rangle. \sigma(42 \mapsto _)$ in the second triple, as address 42 is already allocated. Consequently, framing yields an invalid triple with a satisfiable precondition (e.g., a singleton set with a state σ satisfying $42 \mapsto _$), but an unsatisfiable postcondition.

This problem with the candidate definition is caused by the incompatibility between the non-local nature of allocation and the local reasoning embodied by the frame rule. Existing work on unary reasoning (i.e., not involving hyperproperties) has addressed this problem in two different ways. Ascari et al. [2] fix a set of *syntactic* rules and then prove that this set of rules guarantees the soundness of the proof system. However, this syntactic approach is not easily extensible, as adding a new rule that is sound by itself (w.r.t. the semantic definition of validity) is not guaranteed to preserve the soundness of the overall system. Zilberstein et al. [39] instead instrument the semantics of the programming language with a notion of *allocator* (a function that returns a set of heap locations that can be allocated in a given state), and then define a triple to be valid iff it holds for all allocators. While this approach solves the particular problem with allocation in their setting, it requires instrumenting the semantics of the programming language (with the allocator), and does not provide a general solution for other constructs that cause similar issues with framing, as we explain next.

⁵As we will discuss in Sect. 3.3, some errors (specifically, *domain violation errors*) do not behave this way. Here, we discuss runtime errors such as assertion violations, null dereferences, or division by zero, which would also occur in larger heaps.

Non-local errors. The problem described above is neither specific to allocation nor to reachability reasoning, but also occurs when operations lead to *non-local* errors, that is, errors due to insufficient resources that may not occur in a larger heap. After adding the missing resources with the frame rule, these operations succeed, leading to an incorrect labeling of the resulting states:

$$\frac{\models_0 [\forall \langle \sigma \rangle. \sigma(\text{ok} : \text{emp})] \text{ free}(x) [\forall \langle \sigma \rangle. \sigma(\text{er} : \text{emp})]}{\models_0 \underbrace{[(\forall \langle \sigma \rangle. \sigma(\text{ok} : \text{emp})) \star (\forall \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto _))]}_{\forall \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto _)} \text{ free}(x) \underbrace{[(\forall \langle \sigma \rangle. \sigma(\text{er} : \text{emp})) \star (\forall \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto _))]}_{\forall \langle \sigma \rangle. \sigma(\text{er} : x \mapsto _)}} \text{FRAME}$$

We call such errors, which arise from accessing memory locations that are not currently owned, *domain violation errors*. Besides for deallocation, they may occur for heap read and write operations. In contrast, all other errors (e.g., assertion violations and dereferencing null pointers) are *local* (i.e., if they occur in a state, they also occur in any larger state), and thus are compatible with the frame rule.

Baking in the frame rule. To solve these problems, we take inspiration from existing unary separation logics (e.g., [6, 7]) and adapt it to hyperproperty reasoning: We require the validity of framing *as part of* the definition of triple validity. Intuitively, we define the hyper-triple $[P] C [Q]$ to be valid iff $\models_0 [P \star F] C [Q \star F]$ holds for every admissible⁶ frame F .

This approach yields a simple semantic definition of triple validity that ensures the soundness of the frame rule, even in the presence of reachability reasoning and error states. In particular, it rules out the incorrect derivations above, as both examples start from an *invalid* triple. In the allocation example, the first triple does not preserve the frame $\forall \langle \sigma \rangle. \sigma(\text{ok} : 42 \mapsto _)$; in the de-allocation example, the first triple does not preserve the frame $\forall \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto _)$.

Simplifying the meta theory. The above definition of validity is sufficient to obtain a sound logic. It makes the soundness proof of the frame rule straightforward, but requires one to prove that every other rule yields a valid triple for arbitrarily complex frames F .

We make a key observation to simplify these proofs: It is sufficient for validity to require the preservation of *unary* frames, that is, frames with a single $\forall$ -quantifier. Thus, intuitively, a hyper-triple $[P] C [Q]$ is valid, denoted $\models [P] C [Q]$, iff $\models_0 [P \star (\forall \langle \sigma \rangle. \sigma(\text{ok} : f))] C [Q \star (\forall \langle \sigma \rangle. \sigma(\text{ok} : f))]$ holds for all (admissible) unary frames f .

This weaker definition is sufficient to guarantee the preservation of all $\forall^+$ -frames (and, thus, soundness of the frame rule) for the following reasons:

- (1) Any $\forall^+$ -hyperassertion F can be decomposed into a (potentially infinite) disjunction of unary $\forall$ -assertions $\bigvee_i (\forall \langle \sigma \rangle. \sigma(f_i))$. For example, $\forall \langle \sigma_1 \rangle. \forall \langle \sigma_2 \rangle. \exists n. \sigma_1(x = n) \wedge \sigma_2(x = n)$ is equivalent to $\bigvee_i (\forall \langle \sigma \rangle. \sigma(x = i))$.⁷
- (2) The hyper separating conjunction distributes over (infinite) disjunctions: $P \star \bigvee_i Q_i \equiv \bigvee_i (P \star Q_i)$.
- (3) HSL's triples support the (infinite) disjunction rule. That is, if $\models [P_i] C [Q_i]$ holds for all i then $\models [\bigvee_i P_i] C [\bigvee_i Q_i]$ also holds.

We formalize this argument as part of our soundness proof for the frame rule (see the extended version of this paper [23]). The soundness proof of *every other rule* becomes significantly simpler, as they need to consider only unary $\forall$ -frames.

⁶That is, a frame F that satisfies the premises of the rule **FRAME**, see Sect. 2.3.

⁷Intuitively, this means that any safety hyperproperty can be seen as a (potentially infinite) disjunction of unary safety properties. Reducing safety hyperproperties to unary properties is known from product constructions; the difference here is that we do not enlarge the states, but instead reflect all possible combinations in a (potentially infinite) disjunction.

3.3 Local Reachability Reasoning in the Presence of Errors

Requiring frame preservation as part of triple validity has possibly surprising consequences for reasoning about errors. As explained in the previous subsection, our definition of triple validity *correctly* rejects triples where error states originate only from non-local (domain violation) errors, such as the following triple:

$$[\exists\langle\sigma\rangle. \sigma(\text{ok} : \text{emp})] \text{ free}(x) [\exists\langle\sigma\rangle. \sigma(\text{er} : \text{emp})] \quad (3)$$

which expresses that de-allocating x may lead to an error when executed in an empty heap. This triple is *not* valid because there are frames, such as $\forall\langle\sigma\rangle. \sigma(\text{ok} : x \mapsto _)$, for which the resulting postcondition does *not* hold: adding ownership of the location at x causes the de-allocation to succeed, such that it can no longer reach an error state. Non-local errors are merely an artifact of local reasoning in separation logic, but do not necessarily correspond to actual errors at runtime.

However, our definition of validity also rejects *reasonable* triples, in fact, *any* triple that expresses reachability properties for programs that may cause non-local errors. For example, the triple

$$[\exists\langle\sigma\rangle. \sigma(\text{ok} : x \mapsto _)] \text{ free}(x) [\exists\langle\sigma\rangle. \sigma(\text{ok} : x \mapsto \perp)] \quad (4)$$

expresses a reachability property, which clearly holds: starting from a state where x is allocated, freeing x succeeds and may lead to a state where x is no longer owned (in HSL, de-allocated memory contains a special value $\perp$). However, and perhaps surprisingly, this triple is also *invalid* because it does not preserve the frame $\forall\langle\sigma\rangle. \sigma(\text{ok} : y \mapsto _)$. The reason is subtle: The precondition (including the frame) guarantees that $x \mapsto _ * y \mapsto _$ (and thus $x \neq y$) holds in at least one state; all other initial states are guaranteed only to satisfy $y \mapsto _$. In particular, x and y might alias in some other initial state, such that freeing x loses the ownership of y , which violates the framed conjunct of the postcondition.

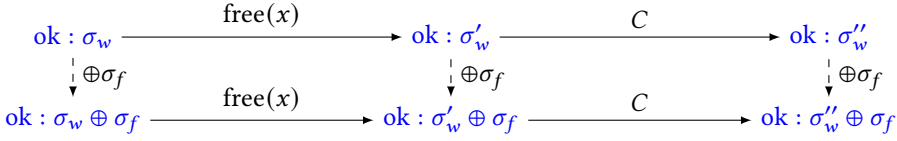
Even if not apparent, this undesired side effect of our definition of triple validity is also caused by non-local errors. To understand why, consider a set with two initial states σ_w and σ_u satisfying the precondition $\exists\langle\sigma\rangle. \sigma(\text{ok} : x \mapsto _)$: σ_w witnesses the existential quantifier, and thus owns x , whereas σ_u does not witness the existential quantifier, and thus is unconstrained. In particular, assume that x and y alias in σ_u (i.e., $x = y$), and that σ_u does not own x . The executions of $\text{free}(x)$ from these two states are illustrated in Fig. 3: The top left of Fig. 3a shows the successful execution from σ_w to σ'_w (where ownership of x is removed), while the top left of Fig. 3b shows the erroneous execution from σ_u (since σ_u does not own x). In the latter case, we record the state in which the (domain violation) error occurred, σ_u , with a special label (which will be explained below).

We now add the frame $\forall\langle\sigma\rangle. \sigma(\text{ok} : y \mapsto _)$, i.e., we add states σ_f with ownership of y to both σ_w and σ_u .⁸ The bottom of Fig. 3a shows that the frame σ_f is preserved when executed from the combined state $\sigma_w \oplus \sigma_f$, as it reaches the state $\sigma'_w \oplus \sigma_f$. In contrast, the bottom of Fig. 3b shows that the frame σ_f is *not* preserved when executed from the combined state $\sigma_u \oplus \sigma_f$: Deallocation now succeeds, and thus ownership of y is removed, as σ_f provides ownership of x (since x and y alias in σ_u).

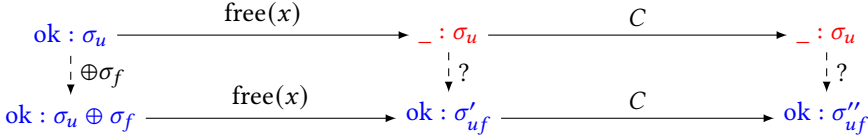
In general, any execution that results in a non-local error might not preserve the frame because (1) resources in the frame have been consumed (like in our example), (2) heap locations have been modified for which the frame provides ownership, or (3) generally because the program executes subsequent statements rather than staying in a failed state (as illustrated by the statement C in Fig. 3a and Fig. 3b).

To account for the fact that frames cannot be expected to be preserved once a non-local error has occurred, we adapt the definitions introduced so far in two ways. First, we allow only those states

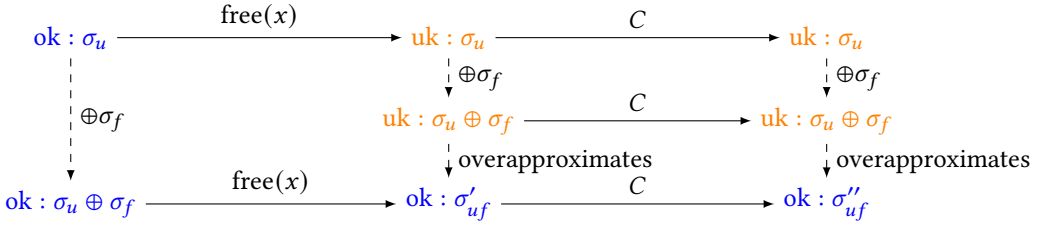
⁸Note that this state σ_f is different between σ_w and σ_u .



(a) Successful execution of de-allocation (top left) and a subsequent statement C (top right). Both preserve frames, as visualized by the downward arrows.



(b) An execution of de-allocation that produces a non-local error (top left). Consequently, the subsequent statement C will *not* be executed and, thus, does not change the state σ_u (top right). We omit the label on the resulting states (in red) as they will be discussed later. The execution at the bottom shows the case where de-allocation (and the subsequent execution of C) succeeds due to a suitable frame. Neither execution preserves frames (indicated by the question marks).



(c) The scenario from Fig. 3b, now correctly labeled as unknown states (in orange). The uk-state σ_u in which the non-local error occurred, extended by the frame, overapproximates all states an execution might reach.

Fig. 3. Illustration of the problem of non-local errors. σ_w denotes a state where x is allocated, and σ_u a state where x is unallocated, but in which $x = y$. σ_f corresponds to a state with ownership of y , that is compatible with σ_w in the first diagram, and with σ_u in the second and third diagram (σ_f is different in each case).

to be labeled as error states in which a *local* error occurred (we will explain below how we label states with non-local errors). As a result, universally-quantified postconditions with ok- or er-labels exclude the occurrence of non-local errors and, thus, are compatible with framing. Second, we adjust triple validity to *relax* the requirement on *existentially-quantified postconditions*⁹ (which do not exclude the possibility of a non-local error), by *effectively* combining the universally-quantified frame only with quantified (ok or er) states, i.e., with all states for universal quantifiers, but only with the witness states for existential quantifiers. If the existential uses an ok- or er-label, those states cannot be affected by non-local errors and, thus, are also compatible with framing.

The first adjustment ensures that triple (3) is correctly deemed invalid. The second adjustment interprets the framed postcondition $\exists \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto \perp) \star \forall \langle \sigma \rangle. \sigma(\text{ok} : y \mapsto _)$ of triple (4) as $\exists \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto \perp * y \mapsto _)$, such that the triple is valid, as it should.

Unknown states. Technically, we handle non-local errors by introducing a third label uk for *unknown* states, in addition to the existing ok and er labels. Executing an operation that causes a non-local error results in an unknown state (whereas local errors result in an error state). This immediately provides the first adaptation discussed above.

⁹Technically, the requirement is relaxed for all postconditions that permit states resulting from non-local errors. In practice, the only such useful postconditions are exactly the existentially-quantified ones.

Intuitively, when a statement leads to a non-local error, the resulting uk-state records the state in which the error occurred. For instance, $_ : \sigma_u$ in Fig. 3b is actually labeled as $\text{uk} : \sigma_u$, as shown in Fig. 3c, both after executing $\text{free}(x)$ and after executing C for any command C . After adding the frame σ_f to σ_u , the de-allocation might or might not lead to a non-local error, depending on whether the frame provides ownership of x . To reflect this uncertainty, we extend the definition of our state-combining predicate $\oplus$ (and thus of our hyper separating conjunction) to yield a uk-label when combining a uk-label (the postcondition) with an ok-label (the frame). For example, $(\text{uk} : \sigma_u) \oplus (\text{ok} : \sigma_f) = \text{uk} : (\sigma_u \oplus \sigma_f)$, as shown in Fig. 3c.

Moreover, executing $\text{free}(x); C$ from the larger state $\sigma_u \oplus \sigma_f$ might lead to a completely different state (for instance, because C modifies variables and allocates new memory), to multiple final states (when C is non-deterministic), or not to any final state (when C does not terminate). This shows that an unknown state $\text{uk} : \sigma$ is an abstraction that may represent zero, one, or multiple concrete states, which may be completely different from σ .¹⁰

Our second adaption above (the relaxed requirement on postconditions) actually corresponds to overapproximating the reachable possible states represented by unknown states, and thus is not overly conservative¹¹. Our triples provide precise information for the standard cases of ok- and er-states; and treat uk-states as overapproximating multiple different concrete states. This allows HSL to prove strong hyperproperties involving successful and erroneous executions (for errors such as accesses to freed memory, null-pointer dereferences, or failed assertions). Our adequacy theorem (Thm. 2) captures this formally: if all state-quantifiers in a hyper-assertion Q are labeled with ok or er, and Q does not contain $\star$, then the relaxation of Q is Q .

Even though uk-states are a coarse abstraction, recording the state in which a non-local error occurred (instead of simply recording *that* a non-local error occurred as with Zilberstein et al. [39]'s *undef* state) is still important: When an unknown state $\text{uk} : \sigma_u$ is combined with an *incompatible* frame σ_f (for instance because both provide ownership of the same location), then it is sound to remove this unknown state from the resulting set of states, as this execution could not have occurred (because the state $\sigma_u \oplus \sigma_f$ before the error is actually inconsistent). In particular, this allows us to define state composition $\oplus$ in a way that is associative, yielding an associative hyper separating conjunction $\star$, and thus improved reasoning principles.

4 Hyper Separation Logic, Formally

In this section, we formally define Hyper Separation Logic (HSL), including its underlying semantic model and proof rules. All formal definitions and theorems presented in this section have been mechanized and proven [22] in Isabelle/HOL [31].

4.1 Programming Language and Semantics

We define Hyper Separation Logic for a simple language with the following commands:

$$C \triangleq \text{skip} \mid x := e \mid x := \text{nonDet}() \mid \text{assume } b \mid \text{assert } b \\ \mid x := \text{alloc}() \mid [x] := e \mid y := [x] \mid \text{free}(x) \mid (C; C) \mid (C + C) \mid C^*,$$

where x and y are non-negative¹² integer program variables, e ranges over the usual non-negative integer expressions, and b is a boolean expression.

¹⁰We can only guarantee that these concrete states agree with σ on the variables that do not syntactically appear in C .

¹¹Conservative in the sense that it forgets all information (except the values of logical variables) about uk states. Not overly conservative in the sense that no information about ok and er states is lost.

¹²Restricted to non-negative values for simplicity, avoiding a separate type distinction between integers and heap locations. This is not a fundamental limitation, as integers and their operations can be encoded using non-negative ones.

skip, assignment $x := e$, and sequential composition are standard. The command $x := \text{nonDet}()$ assigns a non-deterministic unsigned integer value to x . The **assume** b command discards the execution if the current program state does not satisfy b , otherwise it acts like **skip**. The **assert** b command aborts the execution when b is not satisfied, and otherwise behaves like **skip**. The command $x := \text{alloc}()$ allocates a heap location and stores it in x , $[x] := e$ updates a heap location, $y := [x]$ reads from it, and $\text{free}(x)$ deallocates the memory pointed to by x . The $+$ command is nondeterministic choice and $*$ is nondeterministic iteration. Using these commands we can define the standard control structures and constrained non-determinism as follows:

$$\begin{aligned} \text{if } b \text{ then } C_1 \text{ else } C_2 \text{ fi} &\triangleq (\text{assume } b; C_1) + (\text{assume } \neg b; C_2) \\ \text{while } b \text{ do } C \text{ od} &\triangleq (\text{assume } b; C)^*; \text{assume } \neg b \\ x := \text{randInt}(l, u) &\triangleq x := \text{nonDet}(); \text{assume } l \leq x \leq u \end{aligned}$$

A *program state* $\sigma = \langle s, h \rangle$ is a pair consisting of a program store s (a total function from program variables in PVars to $\mathbb{N}$) and a program heap h (a finite partial function from heap locations in $\mathbb{N}$ to $\mathbb{N} \cup \{\perp\}$). Heap location 0 represents a null-pointer, and value $\perp \notin \mathbb{N}$ indicates that a memory location has been freed: Following ISL [34] and OSL [39], we distinguish between un-allocated ($x \notin \text{Dom}(h)$) and freed ($h(x) = \perp$) memory, and we also rule out the re-allocation of freed memory¹³. Arithmetic expressions e are total functions from program stores to $\mathbb{N}$, and boolean expressions b are total functions from program stores to booleans.

A *program configuration* is defined as $c ::= \langle C, \sigma \rangle \mid \mathbf{Aborts}_{\text{dv}} \mid \mathbf{Aborts}_{\text{er}}$, where C is a command, σ is a program state, and $\mathbf{Aborts}_{\text{dv}}$ and $\mathbf{Aborts}_{\text{er}}$ are error configurations. The *small-step semantics* with judgment $c \rightarrow c'$ is defined in the extended version of this paper [23]. The reflexive and transitive closures of $\rightarrow$ is denoted by $\rightarrow^*$. Our semantics models failing executions as transitions that end in an error configuration. As motivated in Sect. 3.3, we distinguish two kinds of failures: (1) accesses to un-allocated memory, which we call *domain violations*, and (2) all other errors, which include accesses to freed memory, null-pointer dereferences, and failed assertions.

4.2 States, Hyper-Assertions, and Hyper Separating Conjunction

The hyper separating conjunction, which we presented in Sect. 2.2, works at the level of *hyper-assertions*, i.e., sets of extended states. *Extended states* are program states with two additional pieces of information. First, we mark the states with *ok*, *er*, or *uk* labels to indicate whether the corresponding execution is normal, erroneous, or unknown, respectively, as explained in Sect. 3. Second, we equip extended states with a *logical store* to record the values of *logical variables*, which remain fixed across executions. Beyond their usual role of recording the initial values of program variables, logical variables play a crucial role in relational reasoning (e.g., to distinguish different executions, as explained by Dardinier and Müller [9]), and, as we will see in Sect. 4.3, they are treated differently from program variables by the relaxation of postconditions.

We thus define an (*extended*) *state* ω as a triple $\langle \Lambda, \sigma_\epsilon \rangle$ consisting of a *logical store* Λ (a total function from logical variables in LVars to $\mathbb{N}$), an execution label $\epsilon \in \{\text{ok}, \text{er}, \text{uk}\}$, and a *program state* σ . *Assertions* are sets of states, and *hyper-assertions* are sets of assertions.

To define the meaning of the (hyper) separating conjunction, we first define the *state-combining predicate* $\oplus$, building on the label combination principles motivated in Sect. 3, as follows:

$$\langle \Lambda, (s, h)_\epsilon \rangle = \langle \Lambda_1, (s_1, h_1)_{\epsilon_1} \rangle \oplus \langle \Lambda_2, (s_2, h_2)_{\epsilon_2} \rangle \stackrel{\text{def}}{\iff} s = s_1 = s_2 \wedge h = h_1 \uplus h_2 \wedge \epsilon = \epsilon_1 \bowtie \epsilon_2 \wedge \Lambda = \Lambda_1 = \Lambda_2$$

where the *label combination operator* $\bowtie$ is defined as follows: $\epsilon_1 \bowtie \epsilon_2$ is *uk* if either ϵ_1 or ϵ_2 is *uk* (propagating the lack of knowledge about the execution status); otherwise it is *er* if either ϵ_1 or ϵ_2

¹³This design provides a practical balance, enabling address arithmetic, while keeping the **ALLOC** rule simple and tractable.

is er (as ok frames can be added to erroneous executions without changing their erroneous status); and it is ok if both ϵ_1 and ϵ_2 are ok (corresponding to the standard case).¹⁴ The (*standard*) *separating conjunction*, $*$, is defined in the usual way: $p * q \triangleq \{\omega \mid \omega_p \in p \wedge \omega_q \in q \wedge \omega = \omega_p \oplus \omega_q\}$. The usual properties of $*$ such as unit element $\{\langle \Lambda, \langle s, \emptyset \rangle_{\text{ok}} \mid \text{true} \rangle\}$, commutativity, associativity and distributivity over (finite and infinite) disjunction are preserved by this extension.

As explained in Sect. 3.1, we define the hyper separating conjunction $\star$ as follows:

$$\begin{aligned} \text{plw}(S, S_1, S_2) &\stackrel{\text{def}}{\iff} \forall \omega_1 \in S_1. \exists \omega \in S. \exists \omega_2 \in S_2. \omega = \omega_1 \oplus \omega_2 \stackrel{\text{def}}{\iff} \text{prw}(S, S_2, S_1) \\ P \star Q &\triangleq \{S \mid S \subseteq S_P * S_Q \wedge S_P \in P \wedge S_Q \in Q \wedge \text{plw}(S, S_P, S_Q) \wedge \text{prw}(S, S_P, S_Q)\} \end{aligned}$$

where the predicates plw and prw ensure that witnesses are preserved when combining states.

The hyper separating conjunction shares many of the key properties of the standard separating conjunction, including the existence of a unit element¹⁵, commutativity, associativity, and distributivity over both finite and infinite unions (which is crucial to prove soundness of the frame rule, as explained in Sect. 3.2).¹⁶ Moreover, standard and hyper separating conjunctions are closely related, as shown by the following properties:

$$\begin{aligned} (\forall \langle \sigma \rangle. \sigma(\epsilon_1 : p)) \star (\forall \langle \sigma \rangle. \sigma(\epsilon_2 : q)) &\equiv \forall \langle \sigma \rangle. \sigma(\epsilon_1 \bowtie \epsilon_2 : p * q) \\ (\exists \langle \sigma \rangle. \sigma(\epsilon_1 : p)) \star (\forall \langle \sigma \rangle. \sigma(\epsilon_2 : q)) &\models \exists \langle \sigma \rangle. \sigma(\epsilon_1 \bowtie \epsilon_2 : p * q). \end{aligned} \tag{5}$$

4.3 Relaxing Postconditions with Unknown States

As discussed informally in Sect. 3.3, framing heap-dependent triples expressing reachability, such as (4), is generally unsound. The reason is that frame disjointness is guaranteed solely in the witness state(s); hence, the frame may be altered in other states. We address this by *relaxing* the postcondition so that the frame applies only to the witness state(s). To formalize this concept, we begin by defining overapproximations of states and assertions.

As motivated in Sect. 3.3, unknown states, which come from domain violation errors, represent our lack of knowledge about corresponding executions in larger states (i.e., after a frame has been added). For example, executing the command $\text{free}(x)$ in the initial state $\text{ok} : \sigma_u$ (at the top of Fig. 3c) leads to a domain violation error because the heap location at x is unallocated. We thus label this state $\text{uk} : \sigma_u$ after the execution of $\text{free}(x)$. After adding the frame $\text{ok} : \sigma_f$, we get the state $\text{uk} : \sigma_u \oplus \sigma_f$, which represents our lack of knowledge about the execution of $\text{free}(x)$; C in the initial state $\text{ok} : \sigma_u \oplus \sigma_f$ (shown at the bottom of Fig. 3c): This execution might lead to an unrelated ok state, to an unrelated er state (e.g., if C tries to deallocate a null pointer), to no state at all (divergence), or to multiple states (non-determinism).

We formally capture this intuition as follows. We say that a state $\langle \Lambda, \sigma_\epsilon \rangle$ *overapproximates* a state $\langle \Lambda', \sigma'_{\epsilon'} \rangle$, written $\langle \Lambda, \sigma_\epsilon \rangle \sqsupseteq \langle \Lambda', \sigma'_{\epsilon'} \rangle$, iff (a) $\epsilon = \text{uk}$ and $\Lambda = \Lambda'$, or (b) $\epsilon \in \{\text{ok}, \text{er}\}$ and $\langle \Lambda, \sigma_\epsilon \rangle = \langle \Lambda', \sigma'_{\epsilon'} \rangle$. As condition (a) reflects, unknown states overapproximate *any* state sharing the same logical store: while the heap update may lead to arbitrary outcomes, program statements cannot change logical variables.¹⁷ In contrast, heap-extended executions for ok and er outcomes are fully determined; thus, they only “overapproximate” themselves, as reflected by condition (b).

¹⁴While HSL allows framing only with ok-states, the four combinations not involving ok-states are relevant for future extensions, in particular, to concurrency.

¹⁵Interestingly, the unit element in this setting is $\mathcal{P}(\{\langle \Lambda, \langle s, \emptyset \rangle_{\text{ok}} \mid \text{true} \rangle\})$, i.e., the power set of the unit element of the standard separating conjunction. As we will see, having a unit element is crucial to prove our adequacy theorem, which requires eliminating the frame in the triple’s validity.

¹⁶Formal proofs of these properties are available in our Isabelle development.

¹⁷Preserving logical stores lets us retain logical variables that capture information relevant for relational reasoning (e.g., to distinguish different executions, as discussed by Dardinier and Müller [9]). Alternatively, one could preserve program variables not modified by C and use them in place of logical variables, but this proves too cumbersome in practice.

Overapproximation extends naturally to sets of states: A set S_0 *overapproximates* a set S , written $S_0 \sqsupseteq S$, iff (A) all ok and er states in S_0 are also in S , i.e., $\{\langle \Lambda, \sigma_\epsilon \rangle \in S_0 \mid \epsilon \in \{\text{ok}, \text{er}\}\} \subseteq S$, and (B) all states in S are overapproximated by some state in S_0 , i.e., $\forall \omega \in S. \exists \omega_0 \in S_0. \omega_0 \sqsupseteq \omega$. Condition (A) ensures that all known states (ok and er) in the overapproximating set S_0 are also present in the original set S , since their outcomes are fully determined, while condition (B) allows extra states to be accounted for by the overapproximation.

Finally, we define *the relaxation of P* , denoted $\mathcal{R}(P)$, as the set of all sets of states S which are overapproximated by some $S_0 \in P$: $\mathcal{R}(P) \triangleq \{S \mid S_0 \in P \wedge S_0 \sqsupseteq S\}$. Under this relaxation, the framed post of (4), $\mathcal{R}((\exists \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto \perp)) \star (\forall \langle \sigma \rangle. \sigma(\text{ok} : y \mapsto _))) = \exists \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto \perp * y \mapsto _)$, adds the frame exclusively to the witness state, achieving the intended effect.

Importantly, existential quantification, e.g., $\exists \langle \sigma \rangle. \sigma(\text{ok} : \dots)$, naturally allows uk states (in contrast to $\forall \langle \sigma \rangle. \sigma(\text{ok} : \dots)$, which excludes them). This inclusion of uk states is essential for the relaxation to soundly overapproximate the program's non-witnessed behaviors. For example, (4) holds because, regardless of which non-witness states appear in the initial set, the relaxation can soundly overapproximate them via appropriate uk states, which are admitted by the existential postcondition.

4.4 Validity of Hyper-Triples

Intuitively, the hyper-triple $[P] C [Q]$ expresses that for every set S of initial states that satisfies P , the resulting set of final states (after executing C in all states from S) satisfies Q . To capture this intuition, we first define the image of a set of states S under the command C , denoted $\text{sem}(C, S)$, as

$$\begin{aligned} & \{ \langle \Lambda, \sigma'_{\text{ok}} \rangle \mid \langle \Lambda, \sigma_{\text{ok}} \rangle \in S \wedge \langle C, \sigma \rangle \rightarrow^* \langle \text{skip}, \sigma' \rangle \} \\ & \cup \{ \langle \Lambda, \sigma'_{\text{er}} \rangle \mid \langle \Lambda, \sigma_{\text{ok}} \rangle \in S \wedge \langle C, \sigma \rangle \rightarrow^* \langle C', \sigma' \rangle \wedge \langle C', \sigma' \rangle \rightarrow \mathbf{Aborts}_{\text{er}} \} \cup \{ \langle \Lambda, \sigma_{\text{er}} \rangle \mid \langle \Lambda, \sigma_{\text{er}} \rangle \in S \} \\ & \cup \{ \langle \Lambda, \sigma'_{\text{uk}} \rangle \mid \langle \Lambda, \sigma_{\text{ok}} \rangle \in S \wedge \langle C, \sigma \rangle \rightarrow^* \langle C', \sigma' \rangle \wedge \langle C', \sigma' \rangle \rightarrow \mathbf{Aborts}_{\text{dv}} \} \cup \{ \langle \Lambda, \sigma_{\text{uk}} \rangle \mid \langle \Lambda, \sigma_{\text{uk}} \rangle \in S \} \end{aligned}$$

Successful executions lead to ok states, whereas erroneous executions lead to either er states (for non-domain violation errors) or uk states (for domain violation errors). Moreover, er and uk states are preserved, but not executed further. Finally, logical stores are preserved across executions.

Using this definition, we can now define the validity of hyper-triples. As explained in Sect. 3, we additionally bake in the preservation of $\forall$ -frames (to ensure the soundness of the frame rule), and relax the postcondition (as explained above) to enable reachability reasoning for heap-manipulating commands. We thus define the validity of a triple as follows:

$$\models [P] C [Q] \stackrel{\text{def}}{\iff} \forall f \in \mathcal{F}(\text{md}(C)). \forall S \in P \star \mathcal{P}(f). \text{sem}(C, S) \in \mathcal{R}(Q \star \mathcal{P}(f))$$

The orange part shows the relaxation of the postcondition, as described above. The green part shows the embedding of the frame rule: The triple must preserve $\forall \langle \sigma \rangle. \sigma(f)$ (represented by $\mathcal{P}(f)$, the powerset of f) for all *admissible* frames f , denoted by $\mathcal{F}(\text{md}(C))$. A frame is *admissible* iff (1) $\text{fv}(f) \cap \text{md}(C) = \emptyset$ holds¹⁸, and (2) f is satisfied by ok states only. Condition (1) is standard, and condition (2) is necessary, since framing with erroneous frames may prevent the execution of C : $\models [\forall \langle \sigma \rangle. \sigma(\text{ok} : x = 0)] x := 5 [\forall \langle \sigma \rangle. \sigma(\text{ok} : x = 5)]$ holds, but framing it with $\forall \langle \sigma \rangle. \sigma(\text{er} : \top)$ prevents all executions of $x := 5$, so the postcondition no longer accurately describes the outcome.

4.5 Rules

We present a selection of HSL's rules in Fig. 4, focusing on rules specific to our separation logic setting. Many more HSL rules, including those to reason about (local) errors, are shown in the extended version of this paper [23]. The rule **FRAME**, crucial for local reasoning, has already been

¹⁸Formally corresponding to $\forall \Lambda, s, s', h, \epsilon. (\forall x \notin \text{md}(C). s(x) = s'(x)) \implies (\langle \Lambda, \langle s, h \rangle_\epsilon \rangle \in f \iff \langle \Lambda, \langle s', h \rangle_\epsilon \rangle \in f)$.

$\frac{\text{FRAME} \quad \begin{array}{c} \text{no intersecting scaffold variables in } Q \text{ and } F \\ \vdash [P] \ C \ [Q] \quad F \models \forall \langle \sigma \rangle. \sigma(\text{ok} : \top) \quad \text{fv}(F) \cap \text{md}(C) = \emptyset \quad \text{no } \exists \langle \cdot \rangle \text{ in } F \end{array}}{\vdash [P \star F] \ C \ [Q \star F]}$		$\frac{\text{JOINTTRUE} \quad \vdash [P] \ C \ [Q]}{\vdash [P \otimes \top] \ C \ [Q \otimes \top]}$	$\frac{\text{SEQ} \quad \begin{array}{c} \vdash [P] \ C_1 \ [R] \quad \vdash [R] \ C_2 \ [Q] \end{array}}{\vdash [P] \ C_1; C_2 \ [Q]}$
$\frac{\text{ALLOC} \quad \begin{array}{c} \text{no } \sigma(\text{er} : _) \text{ in } P \quad x \notin \text{fv}(P) \end{array}}{\vdash [P] \ x := \text{alloc}() \ [(\forall \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto _) \star P)]}$	$\frac{\text{READ} \quad \begin{array}{c} \text{no } \sigma(\text{er} : _) \text{ in } P \quad y \notin \text{fv}(P) \cup \text{pvars}(e) \cup \{x\} \end{array}}{\vdash [(\forall \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto e)) \star P] \ y := [x] \ [(\forall \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto e \wedge y = e)) \star P]}$		
$\frac{\text{WRITE} \quad \text{no } \sigma(\text{er} : _) \text{ in } P}{\vdash [(\forall \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto _) \star P] \ [x] = e \ [(\forall \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto e)) \star P]}$	$\frac{\text{FREE} \quad \text{no } \sigma(\text{er} : _) \text{ in } P}{\vdash [(\forall \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto _) \star P] \ \text{free}(x) \ [(\forall \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto _) \star P)]}$		
$\frac{\text{CONS} \quad \begin{array}{c} \vdash [P] \ C \ [Q] \quad P' \subseteq P \quad Q \subseteq Q' \end{array}}{\vdash [P'] \ C \ [Q']}$	$\frac{\text{WHILESYNC} \quad \begin{array}{c} I \models \forall \langle \sigma_1 \rangle. \forall \langle \sigma_2 \rangle. \sigma_1(\text{ok} : b) \Leftrightarrow \sigma_2(\text{ok} : b) \\ I \models (\forall \langle \sigma \rangle. \sigma(\text{ok} : \top)) \vee (\forall \langle \sigma \rangle. \sigma(\text{er} : \top)) \quad \vdash [(\forall \langle \sigma \rangle. \sigma(\text{ok} : b)) \wedge I] \ C \ [I] \end{array}}{\vdash [I] \ \text{while } b \ \text{do } C \ \text{od} \ [((\forall \langle \sigma \rangle. \sigma(\text{ok} : \neg b)) \wedge I) \vee ((\forall \langle \sigma \rangle. \sigma(\text{er} : \top)) \wedge I) \vee (\forall \langle \sigma \rangle. \perp)]}$		

Fig. 4. Selected Rules of Hyper Separation Logic.

illustrated in Sect. 2.3 and discussed in Sect. 3.2 and above. Its last restriction, forbidding $\exists \langle \cdot \rangle$ in the frame F , is necessary for soundness because reachability in the postcondition implies termination, while C might not always terminate. The scaffold restriction is syntax-specific, explained in Sect. 4.7.

The four rules for basic heap-manipulating commands (**ALLOC**, **READ**, **WRITE**, and **FREE**) are uniformly formulated to internalize a *strengthened* (in two ways) frame rule. First, unlike the frame F in **FRAME**, P can existentially quantify over states, as these commands never diverge. Second, the *syntactic* restriction $\text{no } \sigma(\text{er} : _) \text{ in } P$, which prevents P from mentioning er labels, is weaker and easier to check than the *semantic* ok only restriction of **FRAME**. As a result, these rules may be applied with (internalized) frames P such as $\exists \langle \sigma \rangle. \sigma(\text{ok} : \top)$, while the **FRAME** rule rejects such a frame in two ways: it contains $\exists \langle \cdot \rangle$ and allows non- ok states. As illustrated in Sect. 2, the corresponding $\forall$ -conjunction can be distributed over quantified states, e.g., $(\forall \langle \sigma \rangle. \sigma(\text{ok} : p)) \star (\forall \langle \sigma_1 \rangle. \exists \langle \sigma_2 \rangle. \exists v. \sigma_1(\text{ok} : q_1(v)) \wedge \sigma_2(\text{ok} : q_2(v)))$ is equivalent to $\forall \langle \sigma_1 \rangle. \exists \langle \sigma_2 \rangle. \exists v. \sigma_1(\text{ok} : p \ast q_1(v)) \wedge \sigma_2(\text{ok} : p \ast q_2(v))$.

To apply any of these rules, one must first rewrite the precondition in the form $(\forall \langle \sigma \rangle. \sigma(\text{ok} : p)) \star P$. As we have seen in Sect. 2.2, this rewriting is straightforward for $\forall^+ \exists^*$ -preconditions, as all states are constrained, but is more involved for $\exists^*$ -preconditions. For example, consider applying the rule **FREE** to derive triple (4) from Sect. 3: We cannot rewrite directly our precondition $\exists \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto _)$ as $(\forall \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto _)) \star P$ for some P , because $x \mapsto _$ does not hold in *all* states, only in the witness state. To circumvent this issue, we first rewrite $\exists \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto _)$ as $(\exists \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto _)) \otimes \top$: Intuitively, $P \otimes \top$ expresses that only a *subset* of states has to satisfy P .¹⁹ Since $\otimes \top$ allows us to discard states, we now have the equivalence with $((\forall \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto _)) \star (\exists \langle \sigma \rangle. \sigma(\text{ok} : \top))) \otimes \top$: We discard all states that do not satisfy $x \mapsto _$, retaining only those that do (including the witness). Finally, we can eliminate the $\otimes \top$ using the **JOINTTRUE** rule, as follows:

$$\frac{\frac{\vdash [(\forall \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto _) \star (\exists \langle \sigma \rangle. \sigma(\text{ok} : \top))]}{\vdash [((\forall \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto _) \star (\exists \langle \sigma \rangle. \sigma(\text{ok} : \top))) \otimes \top]} \quad \frac{\vdash [(\forall \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto _) \star (\exists \langle \sigma \rangle. \sigma(\text{ok} : \top))]}{\vdash [(\forall \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto _) \star (\exists \langle \sigma \rangle. \sigma(\text{ok} : \top))]} \quad \text{FREE}}{\vdash [((\forall \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto _) \star (\exists \langle \sigma \rangle. \sigma(\text{ok} : \top))) \otimes \top] \ \text{free}(x) \ [((\forall \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto _) \star (\exists \langle \sigma \rangle. \sigma(\text{ok} : \top))) \otimes \top]} \quad \text{JOINTTRUE}$$

$\underbrace{\quad}_{\exists \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto _)}$
 $\underbrace{\quad}_{\exists \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto _)}$

The same pattern can be applied to other heap-manipulating commands when the precondition is not in the desired form, and to use the rule **FRAME** with $\exists^*$ -preconditions and postconditions. While handling such cases may seem complex at first, these steps are largely routine and could be automated or incorporated into derived rules, so they do not pose a significant practical challenge.

¹⁹Here, $\otimes$ is the join operation $P \otimes Q \triangleq \{S_P \cup S_Q \mid S_P \in P \wedge S_Q \in Q\}$.

The rule **WHILESYNC** is analogous to that of Dardinier and Müller [9], but generalized to account for local errors. It applies to loops whose executions preserve identical control flow across all runs. Note that $\sigma_1(\text{ok} : b) \Leftrightarrow \sigma_2(\text{ok} : b)$ is short for $(\sigma_1(\text{ok} : b) \Rightarrow \sigma_2(\text{ok} : b)) \wedge (\sigma_2(\text{ok} : b) \Rightarrow \sigma_1(\text{ok} : b))$, where $\sigma_1(\text{ok} : b) \Rightarrow \sigma_2(\text{ok} : b)$ is short for $\sigma_1(\text{ok} : \neg b) \vee \sigma_1(\text{er} : \top) \vee \sigma_1(\text{uk} : \top) \vee \sigma_2(\text{ok} : b)$.

4.6 Soundness

We have formalized and proven in Isabelle/HOL that Hyper Separation Logic is sound:

THEOREM 1. *The rules of Hyper Separation Logic, presented in Fig. 4 and in the extended version of this paper [23], are sound with respect to the definition of validity of hyper-triples (Sect. 4.4).*

We discuss the proof in the extended version of this paper [23]. Moreover, while our definition of validity of hyper-triples is rather involved (since we embed all $\forall$ -frames and relax the postcondition), we prove the following adequacy theorem, showing that hyper-triples imply the expected behavior:

THEOREM 2. *If $\models [P] C [Q]$ holds, and the set S of initial states satisfies P (i.e., $S \models P$), then:*

- (1) *The set $\text{sem}(C, S)$ of reachable states satisfies the relaxation of Q (i.e., $\text{sem}(C, S) \models \mathcal{R}(Q)$).*
- (2) *Additionally, if Q does not mention any uk label, contains no $\star$, and constrains its existentially-quantified states to be ok or er ,²⁰ then $\text{sem}(C, S)$ satisfies Q (i.e., $\text{sem}(C, S) \models Q$).*

As explained in Sect. 4.2, we obtain (1) by eliminating the embedded frame using the unit element of $\star$. Point (2) shows that HSL is precise for postconditions that do not mention uk labels, i.e., for hyperproperties involving successful and erroneous executions (for errors such as accesses to freed memory, null-pointer dereferences, or failed assertions); the $\star$ can usually²¹ be eliminated by distributing it over the different connectives (e.g., as done in Sect. 2.2).

4.7 Syntactic Hyper-Assertions and Scaffold Variables

To ease reasoning, we have formalized the following syntax for hyper-assertions:

$$P ::= \top \mid \perp \mid \sigma(\epsilon : p) \mid \exists n. P \mid \forall n. P \mid \exists \langle \sigma \rangle. P \mid \forall \langle \sigma \rangle. P \mid P \vee P \mid P \wedge P \mid P \otimes P \mid P \star P$$

where P ranges over (syntactic) hyper-assertions, σ over state variables in SVars , ϵ over labels $\{\text{ok}, \text{er}, \text{uk}\}$, p over standard separation logic assertions with scaffold variables (as we explain next), and n over program variables²². We define their interpretation in the extended version of this paper [23], and prove [22] many of the equivalences and entailments used throughout the paper, such as $(\forall \langle \sigma \rangle. P) \star (\forall \langle \sigma \rangle. Q) \models \forall \langle \sigma \rangle. (P \star Q)$, $(\forall \langle \sigma \rangle. P) \star (\exists \langle \sigma \rangle. Q) \models \exists \langle \sigma \rangle. (P \star Q)$, or $\sigma(\epsilon_1 : p) \star \sigma(\epsilon_2 : q) \equiv \sigma(\epsilon_1 \bowtie \epsilon_2 : p * q)$; we show more in the extended version of this paper [23]. We have also used this syntax to formalize the syntactic requirements for different rules, for example the no $\sigma(\text{er} : _)$ in P condition in the rules **ALLOC**, **READ**, **WRITE**, and **FREE**, the absence of $\exists \langle \cdot \rangle$ in the frame F of the **FRAME** rule, or the suitability condition for the second point of the adequacy theorem.²³ All rules consider closed hyper-assertions (no free state variables).

Scaffold variables. Intuitively, scaffold variables can be thought of as existentially-quantified logical variables *per state*: A set of states satisfies a hyper-assertion with scaffold variables iff there exists a way to assign values to the scaffold variables *in each state* such that the hyper-assertion holds for the resulting set of states. We formalize this notion in the extended version of this paper [23]. Scaffold variables allow hyper-assertions to be conveniently split into a *non-relational*

²⁰This last restriction is required because $\mathcal{R}(\exists \langle \sigma \rangle. \sigma(\text{ok} : p)) = \exists \langle \sigma \rangle. \sigma(\text{ok} : p)$, but $\mathcal{R}(\exists \langle \sigma \rangle. \top) = \top$.

²¹Similarly to $\star$, $\star$ does not generally distribute over $\wedge$.

²²For simplicity, we represent those as De Bruijn indices [11] in our formalization.

²³In practice, we prove the soundness of these rules under a suitable *semantic* restriction, implied by the *syntactic* ones.

part, constraining individual states, and a *relational* part, relating multiple states. For example, the postcondition of the function *compute* from Sect. 2 can be rewritten with *scaffold variables* δ_o and δ_h as follows (where $e_1(\sigma_1) R e_2(\sigma_2)$, is shorthand for $\exists n. \sigma_1(e_1 = n) \wedge \sigma_2(e_2 R n)$, where $R \in \{=, \neq, <, \dots\}$):

$$\begin{aligned} & \forall \langle \sigma_1 \rangle. \forall \langle \sigma_2 \rangle. \exists \langle \sigma \rangle. \exists u, v. \sigma_1(o \mapsto _ * h \mapsto v) \wedge \sigma(o \mapsto u * h \mapsto v) \wedge \sigma_2(o \mapsto u * h \mapsto _) \\ & = \underbrace{(\forall \langle \sigma \rangle. \sigma(o \mapsto \delta_o * h \mapsto \delta_h))}_{\text{non-relational part}} \star \underbrace{(\forall \langle \sigma_1 \rangle. \forall \langle \sigma_2 \rangle. \exists \langle \sigma \rangle. \sigma(\delta_h) = \sigma_1(\delta_h) \wedge \sigma_2(\delta_o) = \sigma(\delta_o))}_{\text{relational part}} \end{aligned}$$

This formulation has the advantage that ownership assertions need not be duplicated for σ , σ_1 , and σ_2 . The *scaffold variables* δ_o and δ_h represent the values stored at locations o and h , respectively. Their values are preserved by the hyper separating conjunction $\star$, which allows the second conjunct to refer to those values to express the relational property. Not only do scaffold variables allow writing more concise specifications, they also enable stronger reasoning principles, such as:

$$\begin{array}{c} \text{READSCF} \\ \hline \text{no } \sigma(\text{er} : _) \text{ in } P \quad y \notin \text{fv}(P) \quad y \neq x \\ \hline \models [(\forall \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto \delta_x)) \star P] \ y := [x] \ [(\forall \langle \sigma \rangle. \sigma(\text{ok} : x \mapsto \delta_x \wedge y = \delta_x)) \star P] \end{array}$$

Rule **READSCF** generalizes **READ** from Fig. 4 by allowing x to hold an abstract value δ_x , constrained in P , rather than requiring it to point to a specific expression e , making it more expressive. As an example, the triple below can be derived from the consequence rule and **READSCF** (with $P \triangleq \forall \langle \sigma_1 \rangle. \exists \langle \sigma_2 \rangle. \exists n. \sigma_1(\delta_x = n) \wedge \sigma_2(\delta_x = n + 1)$), but it cannot be derived from **READ**:

$$[\forall \langle \sigma_1 \rangle. \exists \langle \sigma_2 \rangle. \exists n. \sigma_1(x \mapsto n) \wedge \sigma_2(x \mapsto n + 1)] \ y := [x] \ [\forall \langle \sigma_1 \rangle. \exists \langle \sigma_2 \rangle. \exists n. \sigma_1(y = n) \wedge \sigma_2(y = n + 1)]$$

Finally, since scaffold variables play a role solely during the interpretation process—serving only as an auxiliary “scaffold”—their values are not required to be preserved from the precondition to the postcondition. This motivates the scaffold restriction in the rule **FRAME**, as otherwise:

$$\frac{\models [\forall \langle \sigma \rangle. \sigma(x \mapsto \delta_x)] \ [x] := 42 \ [\forall \langle \sigma \rangle. \sigma(x \mapsto \delta_x)]}{\models [(\forall \langle \sigma \rangle. \sigma(x \mapsto \delta_x)) \star (\forall \langle \sigma \rangle. \sigma(\delta_x = 5))] \ [x] := 42 \ [(\forall \langle \sigma \rangle. \sigma(x \mapsto \delta_x)) \star (\forall \langle \sigma \rangle. \sigma(\delta_x = 5))]} \text{FRAME}$$

4.8 Examples

To conclude this section, we illustrate the expressiveness of HSL’s rules by showing that the program in Fig. 5 (in black) violates generalized non-interference (GNI), an $\exists\forall$ -hyperproperty, while the program in Fig. 6 satisfies GNI ($\forall\forall\exists$), both lying beyond the reach of existing separation logics. We provide further examples in the extended version of this paper [23], including a proof of the existence of an execution with a maximal value ($\exists\forall$): another property that lies beyond the capabilities of existing separation logics.

The heap-independent program in the middle of Fig. 5 is borrowed from Hyper Hoare Logic [9]; we thus focus on the surrounding heap-manipulating commands. We assume that the high input is stored at location h , and that we have at least two different possibilities for that input value, as expressed by the precondition (where δ_h is a scaffold variable). We first use the rule **READSCF** to read the value pointed by h into variable v_h . Note that the rule **READ** cannot be used directly here, as we do not have an expression e to which h points in the precondition. We then rewrite our hyper-assertion (using the consequence rule) before the deallocation, to apply the rule **FREE**. After the heap-independent part, which can be easily handled with the relevant rules from the extended version of this paper [23] (as shown by Dardinier and Müller [9]), we apply the rules **ALLOC** and **WRITE** in sequence, yielding the negation of GNI: All executions starting with the same high input as σ_1 end up with a low output different from that of σ_2 . In other words, observing σ_2 ’s low output leaks information about σ_2 ’s high input, namely that it is different from σ_1 ’s high input.

$$\begin{aligned}
& [(\forall \sigma. \sigma(h \mapsto \delta_h)) \star ((\exists \langle \sigma_1 \rangle. \exists \langle \sigma_2 \rangle. \sigma_1(\delta_h) \neq \sigma_2(\delta_h)) \wedge (\forall \langle \sigma \rangle. \sigma(\text{ok} : \tau)))] \\
& v_h := [h]; \quad \text{(READSCF)} \\
& [(\forall \sigma. \sigma(h \mapsto \delta_h \wedge v_h = \delta_h)) \star ((\exists \langle \sigma_1 \rangle. \exists \langle \sigma_2 \rangle. \sigma_1(\delta_h) \neq \sigma_2(\delta_h)) \wedge (\forall \langle \sigma \rangle. \sigma(\text{ok} : \tau)))] \\
& \models [(\forall \sigma. \sigma(h \mapsto _)) \star ((\exists \langle \sigma_1 \rangle. \exists \langle \sigma_2 \rangle. \sigma_1(v_h) \neq \sigma_2(v_h)) \wedge (\forall \langle \sigma \rangle. \sigma(\text{ok} : \tau)))] \\
& \text{free}(h); \quad \text{(FREE)} \\
& [(\forall \sigma. \sigma(h \mapsto _) \star ((\exists \langle \sigma_1 \rangle. \exists \langle \sigma_2 \rangle. \sigma_1(v_h) \neq \sigma_2(v_h)) \wedge (\forall \langle \sigma \rangle. \sigma(\text{ok} : \tau)))] \\
& \models [(\exists \langle \sigma_1 \rangle. \exists k_1. \sigma_1(k_1 \leq 9) \wedge \exists \langle \sigma_2 \rangle. \exists k_2. \sigma_2(k_2 \leq 9) \wedge \forall \langle \sigma \rangle. \forall k. \sigma(k \not\leq 9) \vee \sigma(v_h) \neq \sigma_1(v_h) \vee \sigma(v_h + k) \neq \sigma_2(v_h + k_2)) \\
& k := \text{nonDet}(); \text{assume } k \leq 9; v_l := v_h + k; \\
& [\exists \langle \sigma_1 \rangle. \exists \langle \sigma_2 \rangle. \forall \langle \sigma \rangle. \sigma(v_h) \neq \sigma_1(v_h) \vee \sigma(v_l) \neq \sigma_2(v_l)] \\
& l := \text{alloc}(); \quad \text{(ALLOC)} \\
& [(\forall \langle \sigma \rangle. \sigma(x \mapsto _)) \star (\exists \langle \sigma_1 \rangle. \exists \langle \sigma_2 \rangle. \forall \langle \sigma \rangle. \sigma(v_h) \neq \sigma_1(v_h) \vee \sigma(v_l) \neq \sigma_2(v_l))] \\
& [l] := v_l \quad \text{(WRITE)} \\
& [(\forall \langle \sigma \rangle. \sigma(x \mapsto v_l)) \star (\exists \langle \sigma_1 \rangle. \exists \langle \sigma_2 \rangle. \forall \langle \sigma \rangle. \sigma(v_h) \neq \sigma_1(v_h) \vee \sigma(v_l) \neq \sigma_2(v_l))]
\end{aligned}$$

Fig. 5. Proof outline showing that the program in black violates GNI, where the high input is stored at location h , and the low output is stored at the newly-allocated location l . A proof outline for the heap-independent part can be seen in [9].

$$\begin{aligned}
& [(\forall \langle \sigma \rangle. \sigma(\text{list}(h, H))) \star (\forall \langle \sigma_1 \rangle. \forall \langle \sigma_2 \rangle. \sigma_1(\text{len}(H)) = \sigma_2(\text{len}(H)))] \\
& \models [(\forall \langle \sigma \rangle. \sigma(\text{list}(0, \delta_L) * \text{lseg}(h, h, \delta_A) * \text{list}(h, \delta_B) * H = \delta_A \frown \delta_B)) \star I_{rel}] \\
& i := h; l := 0; s := 0; \\
& [I_{own} \star I_{rel}] \\
& \text{while } i \neq 0 \text{ do} \\
& \quad [(\forall \langle \sigma \rangle. \sigma(i \neq 0)) \wedge (I_{own} \star I_{rel})] \\
& \models [(\forall \langle \sigma \rangle. \sigma(i \mapsto \delta)) \star (\forall \langle \sigma \rangle. \sigma(\text{list}(l, \delta_L) * \text{lseg}(h, i, \delta_A) * \delta = \langle \delta_v, \delta_{i'} \rangle * \text{list}(\delta_{i'}, \delta_B) * H = \delta_A \frown [\delta_v] \frown \delta_B)) \star I_{rel}] \\
& \quad p := [i]; \quad \text{(READSCF)} \\
& \quad [(\forall \langle \sigma \rangle. \sigma(i \mapsto \delta \wedge p = \delta)) \star (\forall \langle \sigma \rangle. \sigma(\text{list}(l, \delta_L) * \text{lseg}(h, i, \delta_A) * \delta = \langle \delta_v, \delta_{i'} \rangle * \text{list}(\delta_{i'}, \delta_B) * H = \delta_A \frown [\delta_v] \frown \delta_B)) \star I_{rel}] \\
& \models [(\forall \langle \sigma \rangle. \sigma(\text{list}(l, \delta_L) * \text{lseg}(h, p, \text{snd}, \delta_A) * \text{list}(p, \text{snd}, \delta_B) * H = \delta_A \frown \delta_B)) \star I_{rel}] \\
& \quad s := s + p. \text{fst}; i := p. \text{snd}; \\
& \quad [(\forall \langle \sigma \rangle. \sigma(\text{list}(l, \delta_L) * \text{lseg}(h, i, \delta_A) * \text{list}(i, \delta_B) * H = \delta_A \frown \delta_B)) \star I_{rel}] \\
& \models [(\forall \langle \sigma \rangle. \forall k. \sigma(\text{list}(l, \delta_L) * \text{lseg}(h, i, \delta_A) * \text{list}(i, \delta_B) * H = \delta_A \frown \delta_B)) \star \\
& \quad (\forall \langle \sigma_1 \rangle. \forall k_1. \forall \langle \sigma_2 \rangle. \forall k_2. \exists \langle \sigma \rangle. \exists k. \sigma_1(\text{len}(\delta_B)) = \sigma_2(\text{len}(\delta_B)) \wedge \sigma(H) = \sigma_1(H) \wedge \sigma([s \oplus k] \frown \delta_L) = \sigma_2([s \oplus k_2] \frown \delta_L))] \\
& k := \text{nonDet}(); \\
& [I_{own} \star (\forall \langle \sigma_1 \rangle. \forall \langle \sigma_2 \rangle. \exists \langle \sigma \rangle. \sigma_1(\text{len}(\delta_B)) = \sigma_2(\text{len}(\delta_B)) \wedge \sigma(H) = \sigma_1(H) \wedge \sigma([s \oplus k] \frown \delta_L) = \sigma_2([s \oplus k] \frown \delta_L))] \\
& p := \text{alloc}(); \quad \text{(ALLOC)} \\
& [(\forall \langle \sigma \rangle. \sigma(p \mapsto _)) \star I_{own} \star (\forall \langle \sigma_1 \rangle. \forall \langle \sigma_2 \rangle. \exists \langle \sigma \rangle. \sigma_1(\text{len}(\delta_B)) = \sigma_2(\text{len}(\delta_B)) \wedge \sigma(H) = \sigma_1(H) \wedge \sigma([s \oplus k] \frown \delta_L) = \sigma_2([s \oplus k] \frown \delta_L))] \\
& [p] := \langle s \oplus k, l \rangle; \quad \text{(WRITE)} \\
& [(\forall \langle \sigma \rangle. \sigma(p \mapsto \langle s \oplus k, l \rangle)) \star I_{own} \star (\forall \langle \sigma_1 \rangle. \forall \langle \sigma_2 \rangle. \exists \langle \sigma \rangle. \sigma_1(\text{len}(\delta_B)) = \sigma_2(\text{len}(\delta_B)) \wedge \sigma(H) = \sigma_1(H) \wedge \sigma([s \oplus k] \frown \delta_L) = \sigma_2([s \oplus k] \frown \delta_L))] \\
& \models [(\forall \langle \sigma \rangle. \sigma(\text{list}(p, \delta_L) * \text{lseg}(h, i, \delta_A) * \text{list}(i, \delta_B) * H = \delta_A \frown \delta_B)) \star I_{rel}] \\
& l := p; \\
& [I_{own} \star I_{rel}] \\
& \text{od} \quad \text{(WHILESYNC)} \\
& [((\forall \langle \sigma \rangle. \sigma(i = 0)) \wedge (I_{own} \star I_{rel})) \vee ((\forall \langle \sigma \rangle. \sigma(\text{er} : \tau)) \wedge (I_{own} \star I_{rel})) \vee (\forall \langle \sigma \rangle. \perp)] \\
& \models [(\forall \langle \sigma \rangle. \sigma(\text{list}(l, \delta_L) * \text{list}(h, H))) \star (\forall \langle \sigma_1 \rangle. \forall \langle \sigma_2 \rangle. \exists \langle \sigma \rangle. \sigma(H) = \sigma_1(H) \wedge \sigma(\delta_L) = \sigma_2(\delta_L))]
\end{aligned}$$

Fig. 6. Proof that the program in black satisfies GNI, with the (unaltered) H denoting mathematical sequence, being secret, while its length is public. The predicate $\text{list}(h, H)$ is defines as $\text{lseg}(h, 0, H)$, where the predicate $\text{lseg}(h, h_{int}, H)$ is defined as $(h = h_{int} \wedge H = []) \vee (\exists a, h', H'. h \mapsto \langle a, h' \rangle * \text{lseg}(h', h_{int}, H') * H = [a] \frown H')$. The loop invariant is given by $I_{own} \triangleq \forall \langle \sigma \rangle. \sigma(\text{list}(l, \delta_L) * \text{lseg}(h, i, \delta_A) * \text{list}(i, \delta_B) * H = \delta_A \frown \delta_B)$, separation-conjoined with $I_{rel} \triangleq \forall \langle \sigma_1 \rangle. \forall \langle \sigma_2 \rangle. \exists \langle \sigma \rangle. \sigma_1(\text{len}(\delta_B)) = \sigma_2(\text{len}(\delta_B)) \wedge \sigma(H) = \sigma_1(H) \wedge \sigma(\delta_L) = \sigma_2(\delta_L)$.

Similarly to the proof in Fig. 5, the proof in Fig. 6 focuses on the heap-manipulating commands, as the heap-independent aspects are easily established with the relevant rules from the extended version of this paper [23] (cf. [9] for related examples). We assume that the high input is stored in the contents of the list at h (represented by the mathematical sequence H), while its length is treated as a low-security input. After applying the consequence rule and initializing $i := h$ as a traverse pointer, $l := 0$ for the new list under construction, and $s := 0$ to accumulate the sum of traversed secret values, we ensure that the invariant $I_{own} \star I_{rel}$ holds at entry.

The proof of the loop invariant begins with the standard ownership factorization, followed by the application of `ReadScf` (similarly to Fig. 5, we cannot apply `Read`). We then leverage the fact that scaffold variables are not preserved from pre- to postcondition, renaming δ_A to include the previous δ_A together with the iterated element δ_v . Next, we systematically apply the assign and havoc rules from the extended version of this paper [23], after which `Alloc` is applied. This produces a hyper-assertion in the required form, allowing a straightforward application of `Write`. The invariant proof concludes with a renaming of δ_L and an application of the assign rule from the extended version of this paper [23].

Since the invariant ensures that the loops progress in sync, we can apply the `WhileSync` rule. Its erroneous disjunct, $(\forall \langle \sigma \rangle. \sigma(er : \top)) \wedge (I_{own} \star I_{rel})$, can be dropped as the corresponding invariant is ok-only, while its third disjunct, $\forall \langle \sigma \rangle. \perp$, can be dropped as the invariant is downward closed. We conclude the proof with a final application of `Cons` together with the definitions of list and lseg.

5 Related Work

As discussed throughout the paper, the closest related works are Hyper Hoare Logic (HHL) [9] and Outcome Separation Logic (OSL) [39]. Our work builds on the ideas from HHL, in particular, tracking sets of reachable states and using universal and existential quantification over initial and reachable states to express and prove hyperproperties. However, HSL goes significantly beyond HHL by supporting separation logic reasoning about heap-manipulating programs, which is enabled by its novel hyper separating conjunction and generalized frame rule. Both are substantial generalizations of their standard SL counterparts.

Outcome Separation Logic (OSL) [39] extends Outcome Logic (OL) [38] with separation-logic-style correctness and incorrectness reasoning for heap-manipulating programs. Similarly to HSL, OSL's judgments (in its non-deterministic instantiation) are interpreted over sets of states, and thus both logics face similar challenges. In particular, OSL can express combinations of unary safety and reachability properties, i.e., combinations of $\forall$ - and $\exists$ -properties. However, unlike HSL, OSL cannot express or reason about hyperproperties, i.e., properties relating multiple executions of a program, such as monotonicity and non-interference ($\forall\forall$), GNI ($\forall\forall\exists$), or non-determinism ($\exists\exists$), since these require more than one state quantifier. This difference is also reflected in the design of the separating conjunction and frame rule. A key technical feature of OSL is an *asymmetric* separating conjunction, defined *syntactically*, between an outcome assertion and a standard unary SL assertion. Correspondingly, the frame in OSL's frame rule is restricted to a standard SL assertion f , which intuitively corresponds to the HSL assertion $\forall \langle \sigma \rangle. \sigma(ok : f)$. If adopted in our setting, such an asymmetric syntactic star would restrict framing to unary assertions, and would therefore prevent framing relations between executions. In contrast, HSL uses a *symmetric* hyper separating conjunction, defined *semantically* over sets of labeled states. Accordingly, HSL's frame rule allows $\forall^*$ -frames expressing relational properties, which is crucial for compositional reasoning about hyperproperties, and provides an important foundation for future extensions to concurrency (where the postconditions of parallel threads are themselves hyper-assertions). Finally, OSL models domain violations using a dedicated *undef* state and a special outcome assertion $\top$, whereas HSL tracks

unknown states via a uk label; in our setting, this design is essential to obtain an associative symmetric hyper separating conjunction.

Similarly to HSL and OSL, many logics for reachability have been extended from Hoare logic [17, 24] versions to separation logic (SL) [35] versions, to support local reasoning for heap-manipulating programs, including (Concurrent) Incorrectness Separation Logic (ISL) [33, 34] ($\exists$ -properties, based on Incorrectness Logic [32]), Separation Sufficient Incorrectness Logic (SSIL) [2] ($\exists$ -properties), and Exact Separation Logic (ESL) [25] ($\forall$ and $\exists$ -properties). Unlike HSL, these logics reason only about *unary* properties of programs, and thus defining the separating conjunction in their settings is straightforward. Similarly, *InsecSL* [30] extends *Insecurity Logic* [29], a program logic for proving violations of $\forall\forall$ -properties (i.e., $\exists\exists$ -properties), to a separation logic setting. HSL subsumes SL ($\forall$) and SSIL ($\exists$), in the sense that the triples derivable in these logics are also derivable in HSL. Moreover, ISL ($\exists$), ESL ($\forall$ and $\exists$), and Insec triples ($\exists\exists$) can be expressed as HSL triples, but not with the standard state-quantifiers ($\forall\langle\sigma\rangle$ and $\exists\langle\sigma\rangle$), as shown in Dardinier and Müller [10]. Because of this discrepancy, we have not explored whether their rules can be derived from HSL rules. Additionally, HSL can prove more general reachability properties, such as GNI, that involve both existential and universal quantification; it also allows one to compose different kinds of properties in one proof, for instance, to compose non-interference for a deterministic statement with GNI for a non-deterministic one.

In the domain of safety hyperproperties, Relational Separation Logic (RSL) [37] extends Relational Hoare Logic [4] to reason about $\forall\forall$ -properties of heap-manipulating programs, and LGTM [20] extends LHC [14] (itself extending Cartesian Hoare Logic [36]) to reason about $\forall^*$ properties of heap-manipulating programs. These logics allow proving *relational properties*, i.e., properties relating multiple executions from *different* programs, whereas HSL focuses on *hyperproperties*, i.e., properties relating multiple executions from the *same* program. Moreover, while HSL is able to express and reason about $\forall^*$ -properties, it does not formally subsume RSL or LGTM, because the former enforces the same termination behavior for both executions (either both terminate, or both diverge) and the latter enforces termination for all executions, while HSL does not allow expressing hyperproperties of non-terminating executions. Unlike HSL, which tracks a *set* of reachable states, these logics track k -tuples of executions, and thus defining their separating conjunction (in a pointwise manner) is straightforward, but this limits them to reasoning about k -safety properties only, whereas HSL supports arbitrary quantifier alternation.

Other specialized separation logics have been developed to reason about specific hyperproperties, for example SecCSL [16] and CommCSL [15] target non-interference (i.e., $\forall\forall$) in concurrent programs, while Simuliris [19] and ReLoC [18] target (contextual) refinement (i.e., $\forall\exists$).

6 Conclusion and Future Work

We presented *Hyper Separation Logic* (HSL), the first separation logic that supports reasoning about hyperproperties with arbitrary quantifier alternation, covering both $\forall^*\exists^*$ properties (e.g., generalized non-interference) and $\exists^*\forall^*$ properties (e.g., existence of a maximum). At its core is a *hyper separating conjunction* that composes hyper-assertions by preserving their existential lower bounds and universal upper bounds on states. This connective enables sound, generic rules for heap-manipulating commands and a generalized frame rule for local reasoning, whose soundness follows from embedding all $\forall$ -frames into triple validity. Reachability reasoning is supported by labeled states, including the novel label uk .

Future work includes extending HSL to concurrency, to termination reasoning (both proving and disproving), and to relational properties across multiple programs.

Acknowledgments

We are deeply grateful to Tinko Tinchev for numerous insightful discussions and significant contributions to the conceptual development and technical proofs presented in this work. This research was partially funded by the Ministry of Education and Science of Bulgaria (support for INSAIT, part of the Bulgarian National Roadmap for Research Infrastructure).

Data Availability Statement

All technical results presented in this paper have been formalized and proven in Isabelle/HOL, and our formalization is publicly available [22].

References

- [1] Timos Antonopoulos, Eric Koskinen, Ton Chanh Le, Ramana Nagasamudram, David A. Naumann, and Minh Ngo. 2023. An Algebra of Alignment for Relational Verification. *Proc. ACM Program. Lang.* 7, POPL (Jan. 2023), 20:573–20:603. <https://doi.org/10.1145/3571213>
- [2] Flavio Ascari, Roberto Bruni, Roberta Gori, and Francesco Logozzo. 2024. Sufficient Incorrectness Logic: SIL and Separation SIL. <https://doi.org/10.48550/arXiv.2310.18156> arXiv:2310.18156 [cs]
- [3] Flavio Ascari, Roberto Bruni, Roberta Gori, and Francesco Logozzo. 2025. Revealing Sources of (Memory) Errors via Backward Analysis. *Proc. ACM Program. Lang.* 9, OOPSLA1 (April 2025), 1321–1348. <https://doi.org/10.1145/3720486>
- [4] Nick Benton. 2004. Simple Relational Correctness Proofs for Static Analyses and Program Transformations. In *Proceedings of the 31st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '04)*. Association for Computing Machinery, New York, NY, USA, 14–25. <https://doi.org/10.1145/964001.964003>
- [5] Raven Beutner. 2024. Automated Software Verification of Hyperliveness. In *Tools and Algorithms for the Construction and Analysis of Systems*, Bernd Finkbeiner and Laura Kovács (Eds.). Springer Nature Switzerland, Cham, 196–216. https://doi.org/10.1007/978-3-031-57249-4_10
- [6] Lars Birkedal, Bernhard Reus, Jan Schwinghammer, and Hongseok Yang. 2008. A Simple Model of Separation Logic for Higher-Order Store. In *Automata, Languages and Programming*, Luca Aceto, Ivan Damgård, Leslie Ann Goldberg, Magnús M. Halldórsson, Anna Ingólfssdóttir, and Igor Walukiewicz (Eds.). Springer, Berlin, Heidelberg, 348–360. https://doi.org/10.1007/978-3-540-70583-3_29
- [7] Arthur Charguéraud. 2020. Separation Logic for Sequential Programs (Functional Pearl). *Proc. ACM Program. Lang.* 4, ICFP (Aug. 2020), 1–34. <https://doi.org/10.1145/3408998>
- [8] Michael R. Clarkson and Fred B. Schneider. 2008. Hyperproperties. In *2008 21st IEEE Computer Security Foundations Symposium*. 51–65. <https://doi.org/10.1109/CSF.2008.7>
- [9] Thibault Dardinier and Peter Müller. 2024. Hyper Hoare Logic: (Dis-)Proving Program Hyperproperties. *Proc. ACM Program. Lang.* 8, PLDI (June 2024), 207:1485–207:1509. <https://doi.org/10.1145/3656437>
- [10] Thibault Dardinier and Peter Müller. 2024. Hyper Hoare Logic: (Dis-)Proving Program Hyperproperties (Extended Version). *Proc. ACM Program. Lang.* 8, PLDI (June 2024), 1485–1509. <https://doi.org/10.1145/3656437> arXiv:2301.10037 [cs]
- [11] N. G de Bruijn. 1972. Lambda Calculus Notation with Nameless Dummies, a Tool for Automatic Formula Manipulation, with Application to the Church-Rosser Theorem. *Indagationes Mathematicae (Proceedings)* 75, 5 (Jan. 1972), 381–392. [https://doi.org/10.1016/1385-7258\(72\)90034-0](https://doi.org/10.1016/1385-7258(72)90034-0)
- [12] Edsko de Vries and Vasileios Koutavas. 2011. Reverse Hoare Logic. In *Software Engineering and Formal Methods*, Gilles Barthe, Alberto Pardo, and Gerardo Schneider (Eds.). Springer, Berlin, Heidelberg, 155–171. https://doi.org/10.1007/978-3-642-24690-6_12
- [13] Robert Dickerson, Qianchuan Ye, Michael K. Zhang, and Benjamin Delaware. 2022. RHLE: Modular Deductive Verification of Relational $\exists$ forall $\exists$ Properties. In *Programming Languages and Systems*, Ilya Sergey (Ed.). Springer Nature Switzerland, Cham, 67–87. https://doi.org/10.1007/978-3-031-21037-2_4
- [14] Emanuele D'Ousualdo, Azadeh Farzan, and Derek Dreyer. 2022. Proving Hypersafety Compositionally. *Proc. ACM Program. Lang.* 6, OOPSLA2 (Oct. 2022), 135:289–135:314. <https://doi.org/10.1145/3563298>
- [15] Marco Eilers, Thibault Dardinier, and Peter Müller. 2023. CommCSL: Proving Information Flow Security for Concurrent Programs Using Abstract Commutativity. *Proc. ACM Program. Lang.* 7, PLDI (June 2023), 175:1682–175:1707. <https://doi.org/10.1145/3591289>
- [16] Gidon Ernst and Toby Murray. 2019. SecCSL: Security Concurrent Separation Logic. In *Computer Aided Verification*, Isil Dillig and Serdar Tasiran (Eds.). Vol. 11562. Springer International Publishing, Cham, 208–230. https://doi.org/10.1007/978-3-030-25543-5_13
- [17] Robert W. Floyd. 1967. Assigning Meanings to Programs. *Proceedings of Symposium in Applied Mathematics* (1967), 19–32.

- [18] Dan Frumin, Robbert Krebbers, and Lars Birkedal. 2018. ReLoC: A Mechanised Relational Logic for Fine-Grained Concurrency. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS '18)*. Association for Computing Machinery, New York, NY, USA, 442–451. <https://doi.org/10.1145/3209108.3209174>
- [19] Lennard Gäher, Michael Sammler, Simon Spies, Ralf Jung, Hoang-Hai Dang, Robbert Krebbers, Jeehoon Kang, and Derek Dreyer. 2022. Simuliris: A Separation Logic Framework for Verifying Concurrent Program Optimizations. *Proc. ACM Program. Lang.* 6, POPL (Jan. 2022), 28:1–28:31. <https://doi.org/10.1145/3498689>
- [20] Vladimir Gladsthein, Qiyan Zhao, Willow Ahrens, Saman Amarasinghe, and Ilya Sergey. 2024. Mechanised Hypersafety Proofs about Structured Data. *Proc. ACM Program. Lang.* 8, PLDI (June 2024), 173:647–173:670. <https://doi.org/10.1145/3656403>
- [21] J. A. Goguen and J. Meseguer. 1982. Security Policies and Security Models. In *1982 IEEE Symposium on Security and Privacy*. 11–11. <https://doi.org/10.1109/SP.1982.10014>
- [22] Trayan Gospodinov, Peter Müller, and Thibault Dardinier. 2026. Hyper Separation Logic (artifact). Zenodo. <https://doi.org/10.5281/zenodo.19091148>
- [23] Trayan Gospodinov, Peter Müller, and Thibault Dardinier. 2026. Hyper Separation Logic (extended version). arXiv:2604.12870 [cs.PL] <https://arxiv.org/abs/2604.12870>
- [24] C. A. R. Hoare. 1969. An Axiomatic Basis for Computer Programming. *Commun. ACM* 12, 10 (Oct. 1969), 576–580. <https://doi.org/10.1145/363235.363259>
- [25] Petar Maksimović, Caroline Cronjäger, Andreas Löw, Julian Sutherland, and Philippa Gardner. 2023. Exact Separation Logic: Towards Bridging the Gap Between Verification and Bug-Finding. In *37th European Conference on Object-Oriented Programming (ECOOP 2023) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 263)*, Karim Ali and Guido Salvaneschi (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 19:1–19:27. <https://doi.org/10.4230/LIPIcs.ECOOP.2023.19>
- [26] Daryl McCullough. 1987. Specifications for Multi-Level Security and a Hook-Up. In *1987 IEEE Symposium on Security and Privacy*. 161–161. <https://doi.org/10.1109/SP.1987.10009>
- [27] D. McCullough. 1988. Noninterference and the Composability of Security Properties. In *Proceedings. 1988 IEEE Symposium on Security and Privacy*. 177–186. <https://doi.org/10.1109/SECPRI.1988.8110>
- [28] J. McLean. 1996. A General Theory of Composition for a Class of "Possibilistic" Properties. *IEEE Transactions on Software Engineering* 22, 1 (Jan. 1996), 53–67. <https://doi.org/10.1109/32.481534>
- [29] Toby Murray. 2020. An Under-Approximate Relational Logic: Heralding Logics of Insecurity, Incorrect Implementation & More. <https://doi.org/10.48550/arXiv.2003.04791> arXiv:2003.04791 [cs]
- [30] Toby Murray, Pengbo Yan, and Gidon Ernst. 2023. Compositional Vulnerability Detection with Insecurity Separation Logic. In *Formal Methods and Software Engineering*, Yi Li and Sofiène Tahar (Eds.). Springer Nature, Singapore, 65–82. https://doi.org/10.1007/978-981-99-7584-6_5
- [31] Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. 2002. *Isabelle/HOL - A Proof Assistant for Higher-Order Logic*. Lecture Notes in Computer Science, Vol. 2283. Springer. <https://doi.org/10.1007/3-540-45949-9>
- [32] Peter W. O'Hearn. 2019. Incorrectness Logic. *Proc. ACM Program. Lang.* 4, POPL (Dec. 2019), 10:1–10:32. <https://doi.org/10.1145/3371078>
- [33] Azalea Raad, Josh Berdine, Hoang-Hai Dang, Derek Dreyer, Peter O'Hearn, and Jules Villard. 2020. Local Reasoning About the Presence of Bugs: Incorrectness Separation Logic. In *Computer Aided Verification*, Shuvendu K. Lahiri and Chao Wang (Eds.). Springer International Publishing, Cham, 225–252. https://doi.org/10.1007/978-3-030-53291-8_14
- [34] Azalea Raad, Josh Berdine, Derek Dreyer, and Peter W. O'Hearn. 2022. Concurrent Incorrectness Separation Logic. *Proc. ACM Program. Lang.* 6, POPL (Jan. 2022), 34:1–34:29. <https://doi.org/10.1145/3498695>
- [35] J.C. Reynolds. 2002. Separation Logic: A Logic for Shared Mutable Data Structures. In *Proceedings 17th Annual IEEE Symposium on Logic in Computer Science*. 55–74. <https://doi.org/10.1109/LICS.2002.1029817>
- [36] Marcelo Sousa and Isil Dillig. 2016. Cartesian Hoare Logic for Verifying K-Safety Properties. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '16)*. Association for Computing Machinery, New York, NY, USA, 57–69. <https://doi.org/10.1145/2908080.2908092>
- [37] Hongseok Yang. 2007. Relational Separation Logic. *Theoretical Computer Science* 375, 1 (May 2007), 308–334. <https://doi.org/10.1016/j.tcs.2006.12.036>
- [38] Noam Zilberstein, Derek Dreyer, and Alexandra Silva. 2023. Outcome Logic: A Unifying Foundation for Correctness and Incorrectness Reasoning. *Proc. ACM Program. Lang.* 7, OOPSLA1 (April 2023), 93:522–93:550. <https://doi.org/10.1145/3586045>
- [39] Noam Zilberstein, Angelina Saliling, and Alexandra Silva. 2024. Outcome Separation Logic: Local Reasoning for Correctness and Incorrectness with Computational Effects. *Proc. ACM Program. Lang.* 8, OOPSLA1 (April 2024), 104:276–104:304. <https://doi.org/10.1145/3649821>

Received 2025-11-14; accepted 2026-04-03